



PROYECTO INTERMODULAR

CFGS Desarrollo de Aplicaciones Web
Modalidad VIRTUAL
Informática y Comunicaciones

Library For Everyone - LIFE SYSTEM

Tutor individual: Eliza de la Hoz García

Tutor dual: Arturo Gómez

Curso: 2025/2026

Fecha de presentación: 04 de Junio 2026

Nombre y apellidos: Juan Domingo Meléndez Peña

Email: Juand.melpen@educa.jcyl.es



Tabla de contenido

Contenido

1	Identificación del proyecto	4
2	Descripción general del proyecto	4
2.1	Objetivos	4
2.1.1	Profesional	4
2.1.2	Personal.....	4
2.2	Entorno de trabajo (tecnologías de desarrollo y herramientas).....	5
2.2.1	Software	5
2.2.2	Hardware.....	6
3	Descripción general del producto.....	6
3.1	Visión general del sistema: funcionalidades básicas, usuarios, límites del sistema y otros sistemas con los que pueda interactuar	6
3.2	Descripción breve de métodos, técnicas o arquitecturas utilizadas.....	7
3.2.1	Base de datos	7
3.2.2	Backend.....	10
3.2.3	Frontend.....	16
3.3	Despliegue de la aplicación indicando plataforma tecnológica, instalación de la aplicación y puesta en marcha.....	22
4	Planificación y presupuesto	24
5	Documentación técnica: análisis, diseño, implementación y pruebas.....	24
5.1	Especificación de requisitos	25
5.2	Análisis del sistema	25
5.3	Diseño del sistema:	25
5.3.1	Diseño de la base de datos	25
5.3.2	Diseño de la interfaz de usuario	26
5.3.3	Diseño de la aplicación.....	35

5.4	Implementación:	35
5.4.1	Entorno de desarrollo	35
5.4.2	Estructura del código	35
5.5	Pruebas.....	37
6	Manuales de usuario	41
6.1	Manual de usuario.....	41
6.2	Manual de instalación	53
7	Conclusiones y posibles ampliaciones	55
8	Bibliografía	56
9	Anexo	56

1 Identificación del proyecto

Este proyecto representa la culminación práctica de los conocimientos en diseño, desarrollo y despliegue web que he adquirido durante los últimos dos años.

La aplicación resultante es un sistema integral de gestión diseñado para bibliotecas municipales e institutos públicos. A través de una interfaz centralizada y controlada por menús de navegación (superior y flotante), la plataforma permite administrar módulos clave: inicio, préstamos, donaciones, inventario de libros, gestión de usuarios y soporte técnico.

El principal desafío técnico de este desarrollo consistió en la integración de un flujo completo para las donaciones ciudadanas con generación de reportes, así como la creación de un panel avanzado para el seguimiento de préstamos y devoluciones por usuario. Los detalles operativos y técnicos se profundizarán en las siguientes secciones.

2 Descripción general del proyecto

2.1 Objetivos

Analizamos los siguientes objetivos:

2.1.1 Profesional

- **Desarrollar un producto de software *End-to-End*:** Abarcar todas las fases del ciclo de vida del desarrollo web, desde la toma de requisitos y el diseño inicial hasta la programación y la puesta en marcha de la aplicación.
- **Implementar una arquitectura desacoplada:** Separar de manera efectiva la lógica de negocio (BackEnd) de la interfaz de usuario (FrontEnd) mediante el diseño, desarrollo y consumo de una API RESTful.
- **Adoptar e integrar nuevas tecnologías:** Investigar y aplicar herramientas, lenguajes o *frameworks* modernos que no dominaba al inicio del proyecto, garantizando una curva de aprendizaje efectiva.
- **Dominar el despliegue en entornos reales:** Profundizar en el proceso de despliegue web en producción, configurando servidores, dominios y garantizando la accesibilidad del sistema en la nube.
- **Diseñar bases de datos eficientes:** Estructurar un modelo de datos relacional robusto que permita gestionar de forma segura e íntegra el inventario de libros, los usuarios, los préstamos y las donaciones.
- **Aplicar buenas prácticas y seguridad:** Asegurar el sistema mediante la implementación de autenticación y autorización, protegiendo las rutas sensibles y garantizando un código limpio y mantenible.

2.1.2 Personal

- **Optimizar la gestión del tiempo y la planificación:** Aplicar técnicas de organización y metodologías de trabajo para cumplir con los hitos de desarrollo de un proyecto extenso de forma autónoma.
- **Desarrollar capacidad de resolución de problemas (*Troubleshooting*):** Aprender a gestionar la frustración y buscar soluciones efectivas frente a los *bugs* o problemas técnicos que surgen al trabajar con tecnologías desconocidas.
- **Materializar una solución funcional y real:** Transformar conocimientos abstractos y teóricos en un producto digital tangible que resuelva un problema real (la gestión de bibliotecas municipales).
- **Fomentar el aprendizaje autodidacta:** Mejorar mi capacidad para leer documentación oficial, foros técnicos y manuales para adquirir conocimientos de manera independiente.
- **Enriquecer mi perfil y portafolio:** Construir un caso de estudio sólido, completo y presentable que sirva como carta de presentación para futuras oportunidades laborales o académicas.
- **Mejorar la empatía con el usuario final:** Prestar especial atención a la Experiencia de Usuario (UX) y a la Interfaz de Usuario (UI), asegurándome de que el panel de control sea intuitivo para los bibliotecarios y administradores.
- **Adquirir el hábito de la documentación:** Mejorar mis habilidades de comunicación escrita al registrar el proceso, las decisiones técnicas y el funcionamiento del código para facilitar futuras actualizaciones.

2.2 Entorno de trabajo (tecnologías de desarrollo y herramientas)

Los medios de los que disponemos para realizar el proyecto son los siguientes:

2.2.1 Software

- IDE: Visual Studio Code – Antigravity – IDE Eclipse 25
- Prototipado: Figma
- Base de Datos: PostgreSQL 17 - PgAdmin 9
- Backend: JAVA 21 – JDK 21 – Spring Framework 3.4
- Frontend: Angular 18 – Angular Component – Angular Material.
- Empaquetado: Docker y NPM

- Despliegue: Docker y Google Cloud Plataforma - GCP
- Repositorio: Git – GitHub Destock

2.2.2 Hardware

- Windows 11
- Procesador Dell 12th Gen Intel(R) Core (TM) i5-1235U (1.30 GHz)
- RAM 16.0 GB
- Tarjeta grafica 128.0 MB

3 Descripción general del producto

3.1 Visión general del sistema: funcionalidades básicas, usuarios, límites del sistema y otros sistemas con los que pueda interactuar

El sistema desarrollado inicia con un login, donde los usuarios gestores y administradores podrán ingresar a la aplicación con su correo o nombre de usuario con su respectiva contraseña. De igual forma existe un usuario super administrador que se encarga de administrar a los gestores y administradores que se puedan crear.

La aplicación funciona en producción (servidores web) y entorno local (servidores locales) tanto para el frontend y Backend, además el gestor de la base de datos que se está utilizando.

Funcionalidades Básicas:

- Registro de nuevos – Lector – Gestor - Administrador
- Acceso a la aplicación – login
- Prestamos: crear, actualizar y lectura de datos.
- Donaciones: crear, actualizar y lectura de datos.
- Gestionar libros y Inventarios: crear, actualizar y lectura de datos.

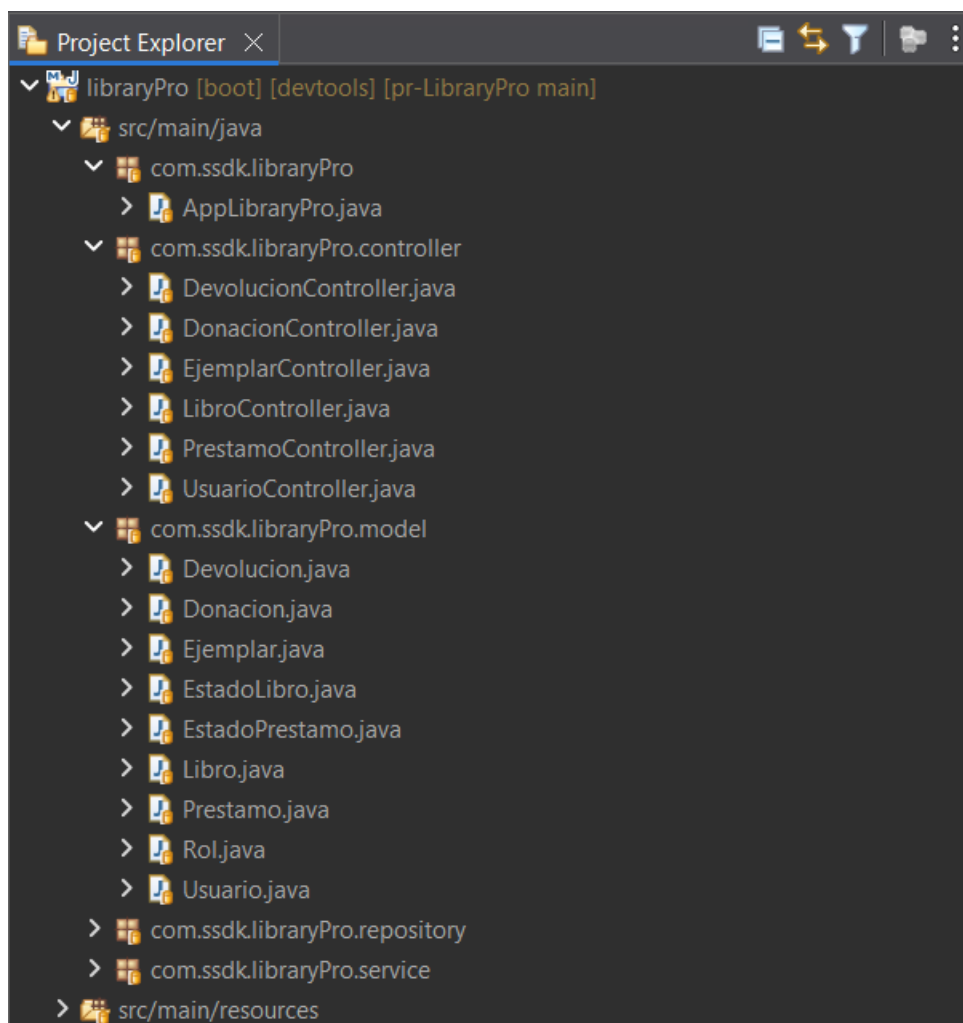

```
DataBase > schema.sql
1  -- Database Schema for LibraryPro - Postgres 17 (Refactored/Normalized)
2  -- Drop tables if they exist to allow clean re-execution
3  DROP TABLE IF EXISTS donacion;
4  DROP TABLE IF EXISTS prestamo;
5  DROP TABLE IF EXISTS ejemplar;
6  DROP TABLE IF EXISTS libro;
7  DROP TABLE IF EXISTS usuario;
8  DROP TABLE IF EXISTS rol;
9  DROP TABLE IF EXISTS estado_libro;
10 DROP TABLE IF EXISTS estado_prestamo;
11
12 CREATE TABLE rol (
13     id SERIAL PRIMARY KEY,
14     titulo VARCHAR(20) UNIQUE NOT NULL
15 );
16
17 CREATE TABLE estado_libro (
18     id SERIAL PRIMARY KEY,
19     descripcion_estado_libro VARCHAR(20) UNIQUE NOT NULL
20 );
21
22 CREATE TABLE estado_prestamo (
23     id SERIAL PRIMARY KEY,
24     descripcion_estado_prestamo VARCHAR(20) UNIQUE NOT NULL
25 );
26
27 -- Create Usuario table
28 CREATE TABLE usuario (
29     id SERIAL PRIMARY KEY,
30     nombre VARCHAR(100) NOT NULL,
31     email VARCHAR(100) UNIQUE NOT NULL,
32     password VARCHAR(255),
33     telefono VARCHAR(20),
34     direccion VARCHAR(255),
35     rol_id INT REFERENCES rol(id),
36     activo BOOLEAN DEFAULT TRUE
37 );
```

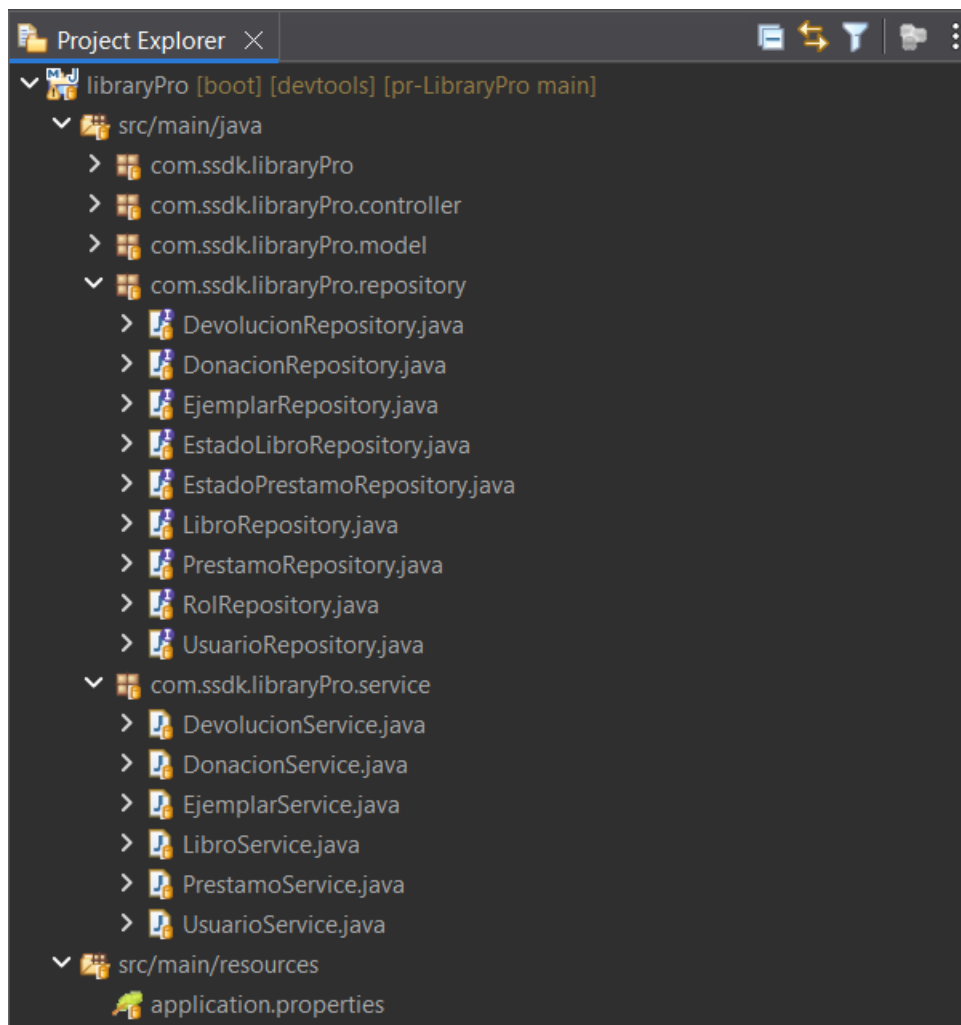
```
DataBase > schema.sql
39 -- Create Libro table
40 CREATE TABLE libro (
41     id SERIAL PRIMARY KEY,
42     isbn VARCHAR(20) UNIQUE,
43     titulo VARCHAR(255) NOT NULL,
44     autor VARCHAR(100) NOT NULL,
45     genero VARCHAR(50)
46 );
47
48 -- Create Ejemplar table
49 CREATE TABLE ejemplar (
50     id SERIAL PRIMARY KEY,
51     libro_id INT REFERENCES libro(id) ON DELETE CASCADE,
52     fec_ingreso DATE NOT NULL DEFAULT CURRENT_DATE,
53     detalle VARCHAR(100),
54     estado_id INT REFERENCES estado_libro(id)
55 );
56
57 -- Create Prestamo table
58 CREATE TABLE prestamo (
59     id SERIAL PRIMARY KEY,
60     fecha_inicio DATE NOT NULL DEFAULT CURRENT_DATE,
61     fecha_devolucion DATE,
62     estado_id INT REFERENCES estado_prestamo(id),
63     usuario_id INT REFERENCES usuario(id) ON DELETE CASCADE,
64     ejemplar_id INT REFERENCES ejemplar(id) ON DELETE CASCADE
65 );
66
67 -- Create Donacion table
68 CREATE TABLE donacion (
69     id SERIAL PRIMARY KEY,
70     fecha DATE NOT NULL DEFAULT CURRENT_DATE,
71     observaciones TEXT,
72     usuario_id INT REFERENCES usuario(id) ON DELETE SET NULL,
73     ejemplar_id INT REFERENCES ejemplar(id) ON DELETE CASCADE
74 );
```

```
DataBase > schema.sql
75 CREATE TABLE devolucion (
76     id SERIAL PRIMARY KEY,
77     prestamo_id INT REFERENCES prestamo(id) ON DELETE CASCADE,
78     fecha_devolucion DATE NOT NULL DEFAULT CURRENT_DATE,
79     gestor_id INT REFERENCES usuario(id),
80     descripcion TEXT
81 );
82
83
```

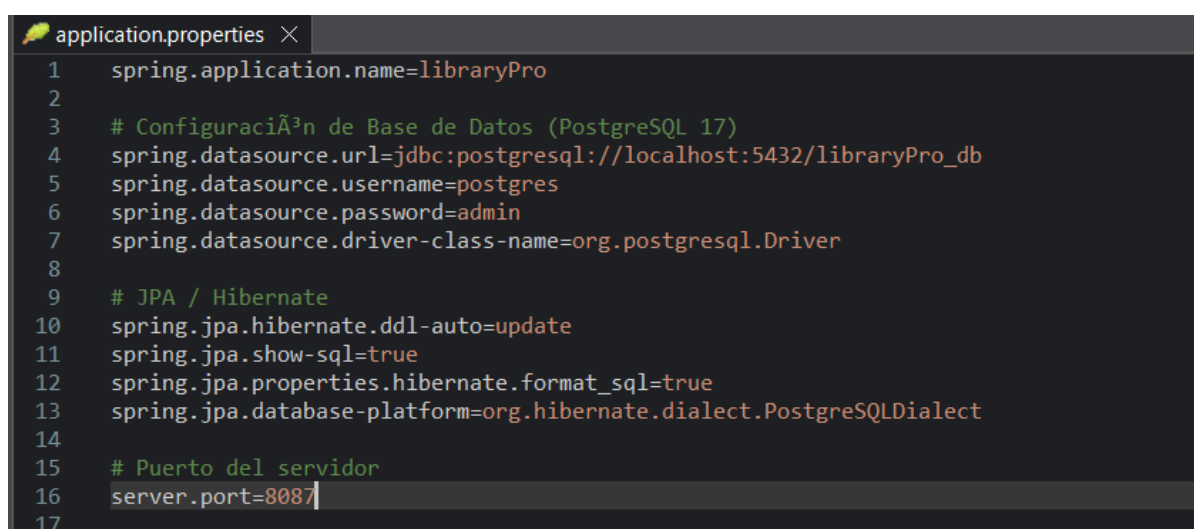
3.2.2 Backend

```
1  ● # Backend - Sistema de Gestión Bibliotecaria
2
3  Este es el backend desarrollado en **Spring Boot**.
4
5  ● ## Estructura de Paquetes
6  - **Model**: Entidades JPA que mapean a las tablas de la DB.
7  - **Repository**: Interfaces que extienden JpaRepository.
8  - **Service**: Clases `@Service` donde reside la lógica de negocio.
9  - **Controller**: Clases `@RestController` que exponen los endpoints de la API.
10
11 ● ## Configuración Inicial
12 Asegúrate de configurar tu `application.properties` con las credenciales de tu base de datos (PostgreSQL17).
```





Contenido del fichero application.properties



Flujo de Datos

Controller (API) → Service (Lógica de negocio/Transacciones) → Repository (Spring Data JPA) →
Hibernate → Base de Datos

Implementación de Muestra: Entidad Usuario

```
Usuario.java ×
1  package com.ssdk.libraryPro.model;
2
3  import jakarta.persistence.*;
4
5  @Entity
6  @Table(name = "usuario")
7  public class Usuario {
8      @Id
9      @GeneratedValue(strategy = GenerationType.IDENTITY)
10     private Long id;
11
12     @Column(nullable = false, length = 100)
13     private String nombre;
14
15     @Column(unique = true, nullable = false, length = 100)
16     private String email;
17
18     @Column(name = "password", length = 255)
19     private String contrasena;
20
21     @Column(length = 20)
22     private String telefono;
23
24     @Column(length = 255)
25     private String direccion;
26
27     @ManyToOne
28     @JoinColumn(name = "rol_id")
29     private Rol rol;
30
31     private Boolean activo = true;
32
33     public Usuario() {
34     }
```



```

36 • public Usuario(Long id, String nombre, String email, String contrasena, String telefono, String direccion, Rol rol,
37     Boolean activo) {
38     this.id = id;
39     this.nombre = nombre;
40     this.email = email;
41     this.contrasena = contrasena;
42     this.telefono = telefono;
43     this.direccion = direccion;
44     this.rol = rol;
45     this.activo = activo != null ? activo : true;
46 }
47
48 • public Long getId() {
49     return id;
50 }
51
52 • public void setId(Long id) {
53     this.id = id;
54 }
55
56 • public String getNombre() {
57     return nombre;
58 }
59
60 • public void setNombre(String nombre) {
61     this.nombre = nombre;
62 }
63
64 • public String getEmail() {
65     return email;
66 }
67
68 • public void setEmail(String email) {
69     this.email = email;
70 }
71
72 • public String getContrasena() {
73     return contrasena;
74 }

```

```

71
72 • public String getContrasena() {
73     return contrasena;
74 }
75
76 • public void setContrasena(String contrasena) {
77     this.contrasena = contrasena;
78 }
79
80 • public String getTelefono() {
81     return telefono;
82 }
83
84 • public void setTelefono(String telefono) {
85     this.telefono = telefono;
86 }
87
88 • public String getDireccion() {
89     return direccion;
90 }
91
92 • public void setDireccion(String direccion) {
93     this.direccion = direccion;
94 }
95
96 • public Rol getRol() {
97     return rol;
98 }
99
100 • public void setRol(Rol rol) {
101     this.rol = rol;
102 }
103
104 • public Boolean getActivo() {
105     return activo;
106 }
107
108 • public void setActivo(Boolean activo) {
109     this.activo = activo;
110 }
111 }

```

Implementación del Repository

```

1 package com.ssd.libraryPro.repository;
2
3 import com.ssd.libraryPro.model.Usuario;
4
5 @Repository
6 public interface UsuarioRepository extends JpaRepository<Usuario, Long> {
7     Optional<Usuario> findByEmail(String email);
8
9     // Método de búsqueda personalizada para nombre, email o teléfono
10    Explain Query with AI
11    @org.springframework.data.jpa.repository.Query("SELECT u FROM Usuario u WHERE LOWER(u.nombre) "
12    + "LIKE LOWER(CONCAT('%', :termino, '%')) "
13    + "OR u.email = :termino OR u.telefono = :termino")
14    java.util.List<Usuario> buscarPorNombreEmailOTelefono(@org.springframework.data.repository.query.Param("termino") String termino);
15
16 }

```

Implementación del Service

```

1 package com.ssd.libraryPro.service;
2
3 import com.ssd.libraryPro.model.Usuario;
4
5 @Service
6 public class UsuarioService {
7     private final UsuarioRepository usuarioRepository;
8
9     public UsuarioService(UsuarioRepository usuarioRepository) {
10         this.usuarioRepository = usuarioRepository;
11     }
12
13     @Transactional(readOnly = true)
14     public List<Usuario> listarTodos() {
15         return usuarioRepository.findAll();
16     }
17
18     @Transactional(readOnly = true)
19     public Optional<Usuario> buscarPorId(Long id) {
20         return usuarioRepository.findById(id);
21     }
22
23     @Transactional(readOnly = true)
24     public Optional<Usuario> buscarPorEmail(String email) {
25         return usuarioRepository.findByEmail(email);
26     }
27
28     @Transactional(readOnly = true)
29     public List<Usuario> buscarUsuario(String termino) {
30         return usuarioRepository.buscarPorNombreEmailOTelefono(termino);
31     }
32
33     @Transactional
34     public Usuario guardar(Usuario usuario) {
35         return usuarioRepository.save(usuario);
36     }
37
38     @Transactional
39     public void eliminar(Long id) {
40         usuarioRepository.deleteById(id);
41     }
42 }

```

Implementación del Controller

```

1 package com.ssd.libraryPro.controller;
2
3 import com.ssd.libraryPro.model.Usuario;
4
5
6 @RestController
7 @RequestMapping("/api/usuarios")
8 @CrossOrigin(origins = "")
9 public class UsuarioController {
10     private final UsuarioService usuarioService;
11
12     public UsuarioController(UsuarioService usuarioService) {
13         this.usuarioService = usuarioService;
14     }
15
16     @GetMapping
17     public List<Usuario> listar() {
18         return usuarioService.listarTodos();
19     }
20
21     @GetMapping("/{id}")
22     public ResponseEntity<Usuario> buscarPorId(@PathVariable Long id) {
23         return usuarioService.buscarPorId(id)
24             .map(ResponseEntity::ok)
25             .orElse(ResponseEntity.notFound().build());
26     }
27
28     @GetMapping("/buscar")
29     public List<Usuario> buscarUsuario(@RequestParam String termino) {
30         return usuarioService.buscarUsuario(termino);
31     }
32
33     @PostMapping
34     public Usuario guardar(@RequestBody Usuario usuario) {
35         return usuarioService.guardar(usuario);
36     }
37
38     @DeleteMapping("/{id}")
39     public ResponseEntity<Void> eliminar(@PathVariable Long id) {
40         usuarioService.eliminar(id);
41         return ResponseEntity.noContent().build();
42     }
43 }

```

Analisis Backend

Verificando las imágenes podemos ver la creación de la Entidad **USUARIO** con sus respectivos atributos, constructor y metodos getters y setters. Luego tenemos la creación de la Interface UsuarioRepository que esta extendida de la clase de JpaRepository.java del paquete org.springframework.data que tiene por interno la configuracion de Hibernate como ORM.

Una vez configurado la conexión a la base de datos con Hibernate tambien tenemos la capa SERVICE, tenemos la creacion de UsuarioService.java que crea la instancia del UsuarioRepository.java, ademas de un constructor

```
public UsuarioService(UsuarioRepository usuarioRepository) {this.usuarioRepository =
usuarioRepository; }
```

tambien podemos comentar que hay metodos propios para utilizar en el core de la aplicación.

Asimismo tenemos creado la capa Repository, donde la clase UsuarioRepository.java se utiliza como la anotacion `@RestController` y `@RequestMapping("/api/usuarios")` que se encarga de mapear los metodos http a la capa del frontend y visualizacion de los datos.

3.2.3 Frontend

```

AppFronD > README_FROND.md M X
AppFronD > README_FROND.md > # Frontend - LIFE SYSTEM > ## Instalación
1  # Frontend - LIFE SYSTEM
2
3  Interfaz de usuario desarrollada en **Angular 21**.
4
5  ## Componentes Implementados
6  - `login`: Acceso al sistema.
7  - `home`: Dashboard principal.
8  - `libro-list`: Buscador y listado de libros.
9  - `prestamo-form`: Gestión de préstamos.
10 - `donacion-form`: Gestión de donaciones.
11 - `usuario-gestion`: Panel centralizado para Responsables (Lectores y Gestores).
12 - `gestor-editar` y `user-editar`: Interfaces de edición estructuradas por nivel de acceso (v1.1.2).
13 - `reportes`: Panel analítico de métricas globales de la biblioteca (v1.1.0).
14 - `soporte`: Vista informativa sobre tecnología y desarrollo (v1.1.0).
15 - `usuario-activity`: Dashboard interactivo de actividad por cada usuario (Préstamos, Devoluciones, Donaciones) (v1.1.3).
16 - `devolucion-form`: Flujo transaccional para el retorno de ejemplares (v1.1.3).
17
18 ## Instalación
19 Ejecuta `npm install` y luego `ng serve` para ver la aplicación.
  
```

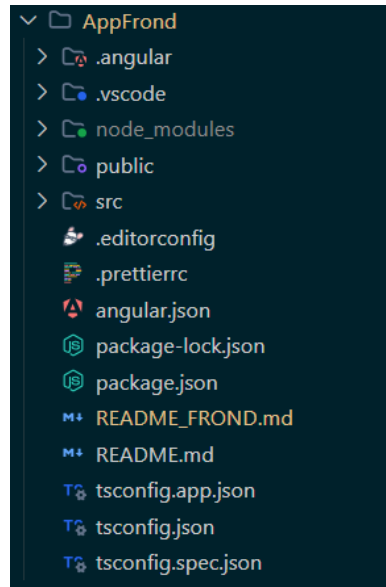
El nuevo bloque de **Arquitectura y Estructura de Proyecto** contiene:

- Una confirmación del **stack tecnológico principal** (Angular 21, TypeScript, HTML y Tailwind CSS).
- Detalles de cada módulo en la carpeta de components/,
detallando 10 subdirectorios funcionales
(/admin, /devoluciones, /donaciones, /inicio, /libros, /login,
/prestamos, /reportes, /soporte, /usuarios).
- Una breve explicación de las responsabilidades de los otros directorios clave:
 - **services/**: Para las peticiones HTTP y conexión al backend.
 - **models/**: Para almacenar las interfaces TypeScript (DTOs).
 - **guards/**: Para la capa de seguridad y protección de tus rutas.
- Además, se describe brevemente cómo operan los archivos raíz que construyen la base (app.routes.ts, app.html y app.ts).

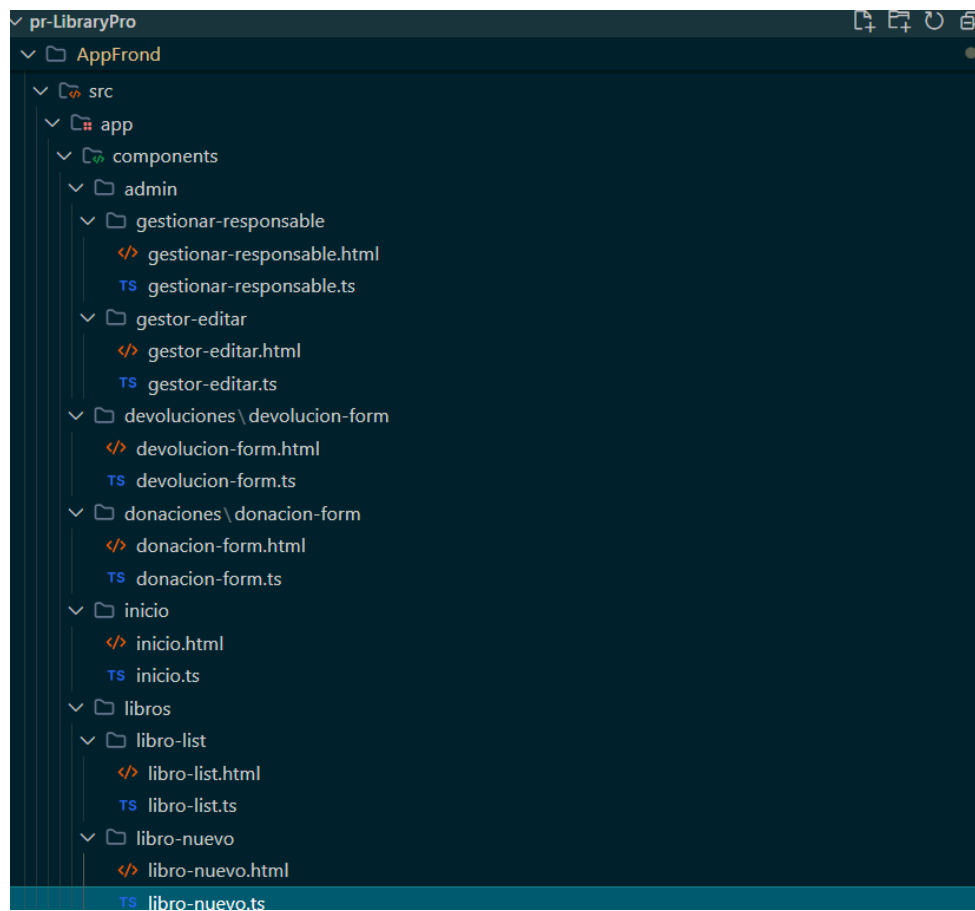
```

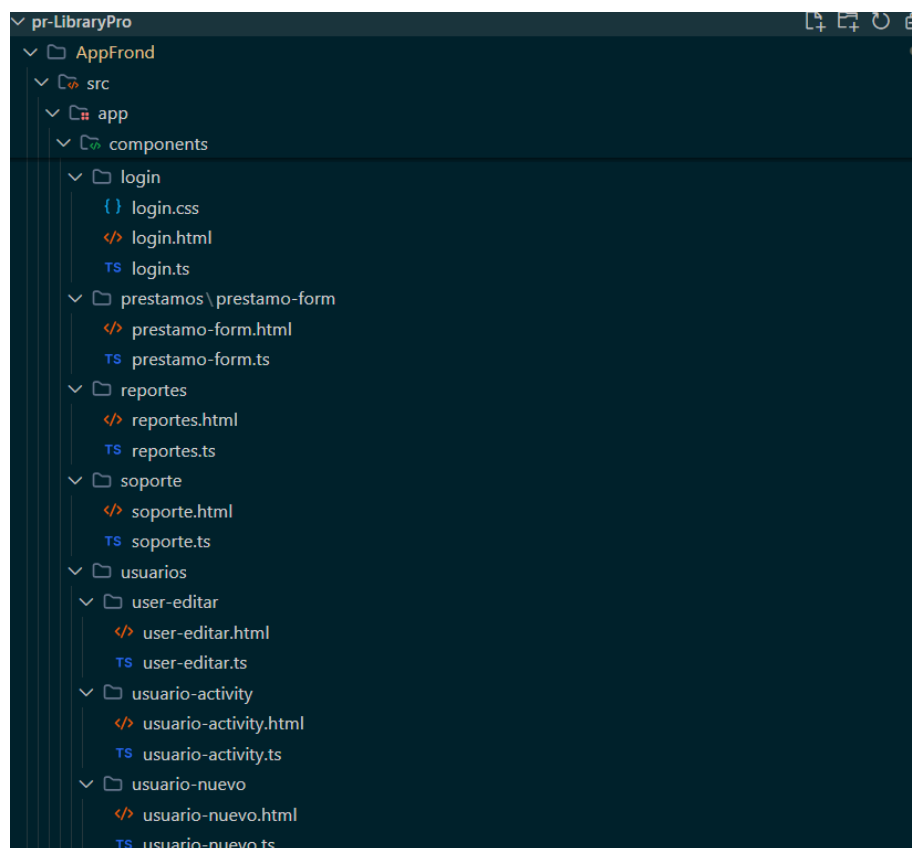
AppFronD > README_FROND.md M X
AppFronD > README_FROND.md > ...
31  ### Organización de Directorios ( `src/app/` )
32
33  - `components/`: Contiene los módulos de la interfaz de usuario, organizados por contexto funcional:
34    - `/admin`: Vistas dedicadas para administradores y configuración de accesos.
35    - `/devoluciones`: Componentes para el flujo de retorno de libros/ejemplares.
36    - `/donaciones`: Formularios y tablas para capturar información de donaciones.
37    - `/inicio`: Dashboard principal u hogar tras el inicio de sesión.
38    - `/libros`: Catálogo interactivo de libros.
39    - `/login`: Vista de autenticación del sistema.
40    - `/prestamos`: Gestión de asignación y tracking de ejemplares prestados.
41    - `/reportes`: Panel analítico de métricas en tiempo real.
42    - `/soporte`: Información técnica y stack del proyecto.
43    - `/usuarios`: Flujos alrededor de cada lector (actividad, alta y edición de perfil).
44
45  - `services/`: Encapsula toda la comunicación asíncrona (Peticiones HTTP) mediante la API REST (Backend Spring Boot). Ejemplo: `auth.service.ts`, `libro.service.ts`, etc.
46
47  - `models/`: Contiene `models.ts`, donde habitan las interfaces tipadas (DTOs) que aseguran la consistencia de los objetos transitados (Ej: Entidad `Usuario`, `Libro`, `Prestamo`).
48
49  - `guards/`: Emplea mecanismos de protección (ej. `auth.guard.ts`) para denegar el paso a componentes privados si el usuario no cuenta con un token de sesión válido.
50
51  - Archivos Raíz:
52    - `app.routes.ts`: Archivo base que define el árbol de navegación del frontend.
53    - `app.html` & `app.ts`: componente principal raíz que estructura elementos fijos (como la barra superior o el Menú *Dock* inferior).
  
```

Directorios del proyecto Frontend

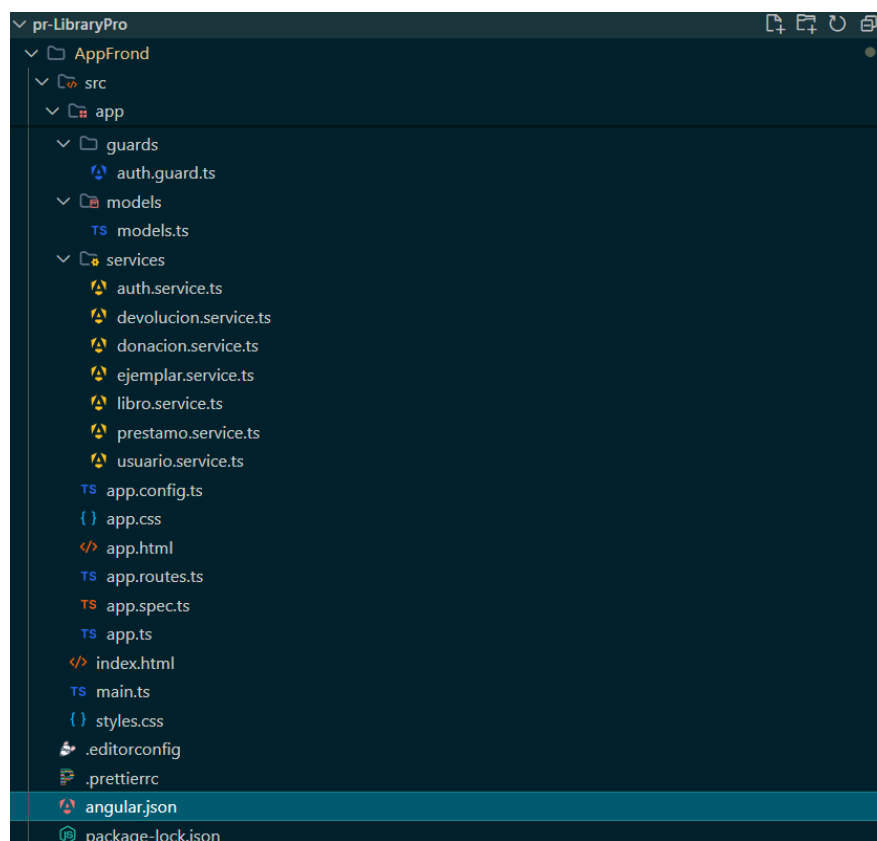


Componentes





Directorios Guards, Models y Services



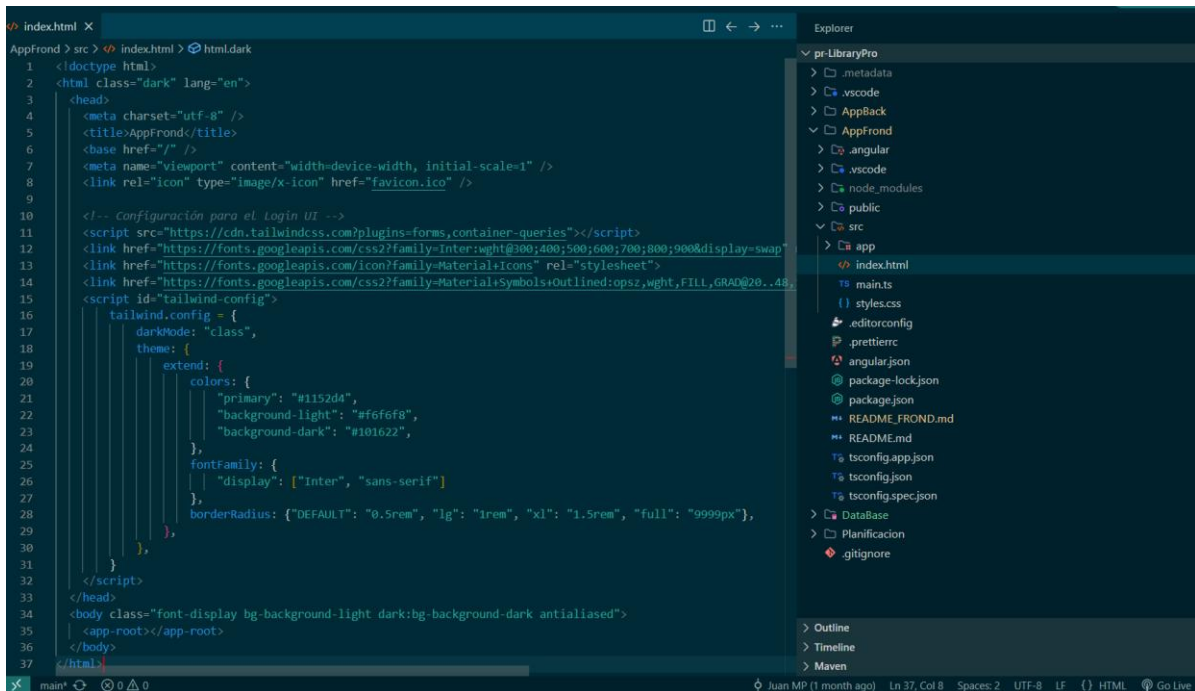
Analisis Frontend

En este analisis vamos a describir como se utiliza y muestra en el component USURIOS la llamada de la API con los datos que nos sirve el Backend mediante los metodos http.

En esta imagen se muestra el src/index.html, es el fichero mas importante en la arquitectura Angular. En el cual se puede verificar en la etiqueta body el elemento app-root

```
34 <body class="font-display bg-background-light dark:bg-background-dark antialiased">
35 |   <app-root></app-root>
36 </body>
```

Que contiene todos los componentes que se utiliza en la aplicación.



```
1 <!doctype html>
2 <html class="dark" lang="en">
3 <head>
4   <meta charset="utf-8" />
5   <title>AppFront</title>
6   <base href="/" />
7   <meta name="viewport" content="width=device-width, initial-scale=1" />
8   <link rel="icon" type="image/x-icon" href="favicon.ico" />
9
10  <!-- Configuración para el Login UI -->
11  <script src="https://cdn.tailwindcss.com?plugins=forms,container-queries"></script>
12  <link href="https://fonts.googleapis.com/css2?family=Inter:wght@300;400;500;600;700;800;900&display=swap"
13  <link href="https://fonts.googleapis.com/icon?family=Material+Icons" rel="stylesheet">
14  <link href="https://fonts.googleapis.com/css2?family=Material+Symbols+Outlined:opsz,wght,FILL,GRAD@20..48,
15  <script id="tailwind-config">
16    tailwind.config = {
17      darkMode: "class",
18      theme: {
19        extend: {
20          colors: {
21            "primary": "#1152d4",
22            "background-light": "#f6f6f8",
23            "background-dark": "#101622",
24          },
25          fontFamily: {
26            "display": ["Inter", "sans-serif"]
27          },
28          borderRadius: {"DEFAULT": "0.5rem", "lg": "1rem", "xl": "1.5rem", "full": "9999px"},
29        },
30      },
31    },
32  </script>
33 </head>
34 <body class="font-display bg-background-light dark:bg-background-dark antialiased">
35   <app-root></app-root>
36 </body>
37 </html>
```

Componente Usuario-nuevo.js

```

1 import { Component } from '@angular/core';
2 import { CommonModule } from '@angular/common';
3 import { FormsModule } from '@angular/forms';
4 import { Router } from '@angular/router';
5 import { UsuarioService } from '../../services/usuario.service';
6 import { Usuario } from '../../models/models';
7
8 @Component({
9   selector: 'app-usuario-nuevo',
10  standalone: true,
11  imports: [CommonModule, FormsModule],
12  templateUrl: './usuario-nuevo.html'
13 })
14 export class UsuarioNuevoComponent {
15   // Inicializamos un modelo de usuario con todos los campos
16   usuario: Usuario = {
17     nombre: '',
18     email: '',
19     telefono: '',
20     direccion: '',
21     contrasena: '',
22     rol: { id: 1, titulo: 'Lector' },
23     activo: true
24   };
25
26   guardando = false;
27
28   constructor(
29     private usuarioService: UsuarioService,
30     private router: Router
31   ) {}
32
33   guardarUsuario() {
34     this.guardando = true;
35     this.usuarioService.guardar(this.usuario).subscribe({
36       next: (res) => {
37         this.guardando = false;

```

Su plantilla Usuario-nuevo.html

```

1 <div class="flex flex-col w-[80%] max-w-2xl mx-auto justify-center gap-4 mb-[25px] mt-1 items-stretch">
2
3   <div class="w-full glass rounded-xl overflow-hidden shadow-2xl border-purple flex flex-col">
4     <div class="h-10 flex items-center px-4 bg-slate-900/60 border-b border-white/20">
5       <div class="flex gap-2">
6         <div class="w-3 h-3 rounded-full bg-red-500 hover:opacity-80 cursor-pointer"></div>
7         <div class="w-3 h-3 rounded-full bg-yellow-500 hover:opacity-80 cursor-pointer"></div>
8         <div class="w-3 h-3 rounded-full bg-green-500 hover:opacity-80 cursor-pointer"></div>
9       </div>
10      <div class="flex-1 text-center text-white text-sm font-semibold mr-10 drop-shadow-md">Crear Nuevo Perfil Lector</div>
11    </div>
12
13    <div class="p-6">
14      <form (ngSubmit)="guardarUsuario()">
15
16        <div class="grid grid-cols-1 md:grid-cols-2 gap-4 mb-2">
17          <!-- Nombre Completo -->
18          <div class="md:col-span-2">
19            <label class="block text-slate-200 text-xs font-bold mb-2 uppercase flex items-center gap-1">
20              Nombre Completo <span class="text-red-400">*</span>
21            </label>
22            <input type="text" name="nombre" [(ngModel)]="usuario.nombre" placeholder="Ej. Juan Pérez" required
23              | class="w-full bg-white/10 border border-white/20 text-white placeholder-white/50 px-4 py-2 rounded-lg focus:outline-none">
24          </div>
25
26          <!-- Email -->
27          <div>
28            <label class="block text-slate-200 text-xs font-bold mb-2 uppercase flex items-center gap-1">
29              correo Electrónico <span class="text-red-400">*</span>
30            </label>
31            <input type="email" name="email" [(ngModel)]="usuario.email" placeholder="correo@ejemplo.com" required
32              | class="w-full bg-white/10 border border-white/20 text-white placeholder-white/50 px-4 py-2 rounded-lg focus:outline-none">
33          </div>
34        </div>

```

Usuarios-services


```

usuario.service.ts X
AppFront > src > app > services > usuario.service.ts > UsuarioService > eliminar
1 import { Injectable } from '@angular/core';
2 import { HttpClient } from '@angular/common/http';
3 import { Observable } from 'rxjs';
4 import { Usuario } from '../models/models';
5
6 @Injectable({
7   providedIn: 'root'
8 })
9 export class UsuarioService {
10   private apiUrl = 'http://localhost:8087/api/usuarios';
11
12   constructor(private http: HttpClient) { }
13
14   listarTodos(): Observable<Usuario[]> {
15     return this.http.get<Usuario[]>(`${this.apiUrl}`);
16   }
17
18   buscarPorId(id: number): Observable<Usuario> {
19     return this.http.get<Usuario>(`${this.apiUrl}/${id}`);
20   }
21
22   guardar(usuario: Usuario): Observable<Usuario> {
23     return this.http.post<Usuario>(this.apiUrl, usuario);
24   }
25
26   buscarUsuario(termino: string): Observable<Usuario[]> {
27     return this.http.get<Usuario[]>(`${this.apiUrl}/buscar?termino=${termino}`);
28   }
29
30   eliminar(id: number): Observable<void> {
31     return this.http.delete<void>(`${this.apiUrl}/${id}`);
32   }
33 }

```

Interface Usuario

```

models.ts X
AppFront > src > app > models > models.ts > Rol
1 export interface Rol {
2   id?: number;
3   titulo: string;
4 }
5
6 export interface EstadoLibro {
7   id?: number;
8   descripcion: string;
9 }
10
11 export interface EstadoPrestamo {
12   id?: number;
13   descripcion: string;
14 }
15
16 export interface Usuario {
17   id?: number;
18   nombre: string;
19   email: string;
20   contrasena?: string;
21   telefono?: string;
22   direccion?: string;
23   rol: Rol;
24   activo: boolean;
25 }
26
27 export interface Libro {
28   id?: number;
29   isbn?: string;
30   titulo: string;
31   autor: string;
32   genero?: string;
33 }
34

```

```

TS app.routes.ts x
AppFronD > src > app > TS app.routes.ts > routes
1 import { Routes } from '@angular/router';
2 import { InicioComponent } from '../components/inicio/inicio';
3 import { LoginComponent } from '../components/login/login';
4 import { LibroListComponent } from '../components/libros/libro-list/libro-list';
5 import { PrestamoFormComponent } from '../components/prestamos/prestamo-form/prestamo-form';
6 import { DonacionFormComponent } from '../components/donaciones/donacion-form/donacion-form';
7 import { GestionarResponsableComponent } from '../components/admin/gestionar-responsable/gestionar-responsable';
8 import { LibroNuevoComponent } from '../components/libros/libro-nuevo/libro-nuevo';
9 import { UsuarioNuevoComponent } from '../components/usuarios/usuario-nuevo/usuario-nuevo';
10 import { ReportesComponent } from '../components/reportes/reportes';
11 import { SoporteComponent } from '../components/soporte/soporte';
12 import { UsuarioActivityComponent } from '../components/usuarios/usuario-activity/usuario-activity';
13 import { DevolucionFormComponent } from '../components/devoluciones/devolucion-form/devolucion-form';
14 import { authGuard } from '../guards/auth.guard';
15
16 export const routes: Routes = [
17   { path: '', redirectTo: '/login', pathMatch: 'full' },
18   { path: 'login', component: LoginComponent },
19   { path: 'home', component: InicioComponent, canActivate: [authGuard] },
20   { path: 'libros', component: LibroListComponent, canActivate: [authGuard] },
21   { path: 'libros/nuevo', component: LibroNuevoComponent, canActivate: [authGuard] },
22   { path: 'prestamos', component: PrestamoFormComponent, canActivate: [authGuard] },
23   { path: 'donaciones', component: DonacionFormComponent, canActivate: [authGuard] },
24   { path: 'gestion-responsables', component: GestionarResponsableComponent, canActivate: [authGuard] },
25   { path: 'usuarios/nuevo', component: UsuarioNuevoComponent, canActivate: [authGuard] },
26   { path: 'usuarios/actividad/:id', component: UsuarioActivityComponent, canActivate: [authGuard] },
27   { path: 'devoluciones/nuevo/:prestamoId', component: DevolucionFormComponent, canActivate: [authGuard] },
28   { path: 'reportes', component: ReportesComponent, canActivate: [authGuard] },
29   { path: 'soporte', component: SoporteComponent, canActivate: [authGuard] }
30 ];
31

```

3.3 Despliegue de la aplicación indicando plataforma tecnológica, instalación de la aplicación y puesta en marcha

Guía de Despliegue Local (Entorno de Desarrollo)

Para ejecutar el proyecto ****LiFe System**** en un entorno local, la arquitectura se divide en tres componentes principales. A continuación, se detallan los pasos para levantar cada uno de los servicios en su orden correcto.

Pre-requisitos

Asegúrate de tener instaladas las siguientes herramientas en tu sistema:

- ****PostgreSQL**** y ****pgAdmin 4**** (Motor y gestor de Base de Datos).
- ****Java JDK**** (Versión 17 o superior recomendada).
- ****Eclipse IDE**** (o cualquier otro IDE compatible con Java como IntelliJ IDEA o VS Code).
- ****Node.js**** y ****npm**** (Entorno de ejecución para el frontend).

1. Configuración de la Base de Datos (pgAdmin)

El sistema utiliza PostgreSQL para la persistencia de los datos.

1. Abre **pgAdmin** y conéctate a tu servidor local.
2. Crea una nueva base de datos para el proyecto (ej. `librarypro_db`).
3. *(Si aplica)* Ejecuta los scripts de la carpeta `DataBase/` para generar las tablas o datos iniciales.
4. Asegúrate de que las credenciales (usuario/contraseña) y la URL de conexión coincidan con las configuradas en el archivo `application.properties` del Backend.

2. Despliegue del Backend (Spring Boot en Eclipse)

El backend es una aplicación Spring Boot, la cual incluye su propio servidor Tomcat embebido, por lo que no requiere configuración de servidores externos.

1. Abre **Eclipse IDE**.
2. Importa el proyecto desde la carpeta `AppBack` como un proyecto Maven.
3. Espera a que Eclipse descargue todas las dependencias del archivo `pom.xml`.
4. Localiza el archivo principal de la aplicación (la clase que contiene el método `main` y la anotación `@SpringBootApplication`).
5. Haz clic derecho sobre este archivo y selecciona
Run As > Java Application
(o **Spring Boot App** si tienes el plugin instalado).
6. El servidor Tomcat embebido se iniciará automáticamente y la API REST quedará disponible, por defecto en `http://localhost:8080`.

3. Despliegue del Frontend (Angular)

La interfaz de usuario consume los servicios levantados en el paso anterior.

1. Abre una terminal y navega a la carpeta del frontend:

```
``bash
```

```
cd AppFron
```

2. Instala las dependencias necesarias (solo la primera vez):

```
bash  
npm install
```

3. Levanta el servidor de desarrollo de Angular ejecutando:

```
bash  
npm run start
```

*(Este comando internamente ejecuta **ng serve**).*

4. Abre tu navegador web y dirígete a <http://localhost:4200> para interactuar con el sistema.

4 Planificación y presupuesto

Las fases del proyecto que nos hemos marcado junto con el tiempo que nos ha llevado cada apartado.

- Análisis (7h)
- Diseño (10h)
- Desarrollo (150h total)
 - o Base de Datos (10h)
 - o BackEnd (50h)
 - o FrontEnd (60h)
 - o Corrección de Bugs (30h)
- Compilación (2h)
- Despliegue (1h)

El tiempo total estimado ascendería a 170 horas.

5 Documentación técnica: análisis, diseño, implementación y pruebas

5.1 Especificación de requisitos

- Windows 11 64-bit 21H2 o superior
- Procesador de 64 Bits.
- BIOS que acepte virtualización
- 16 GB de RAM o más.

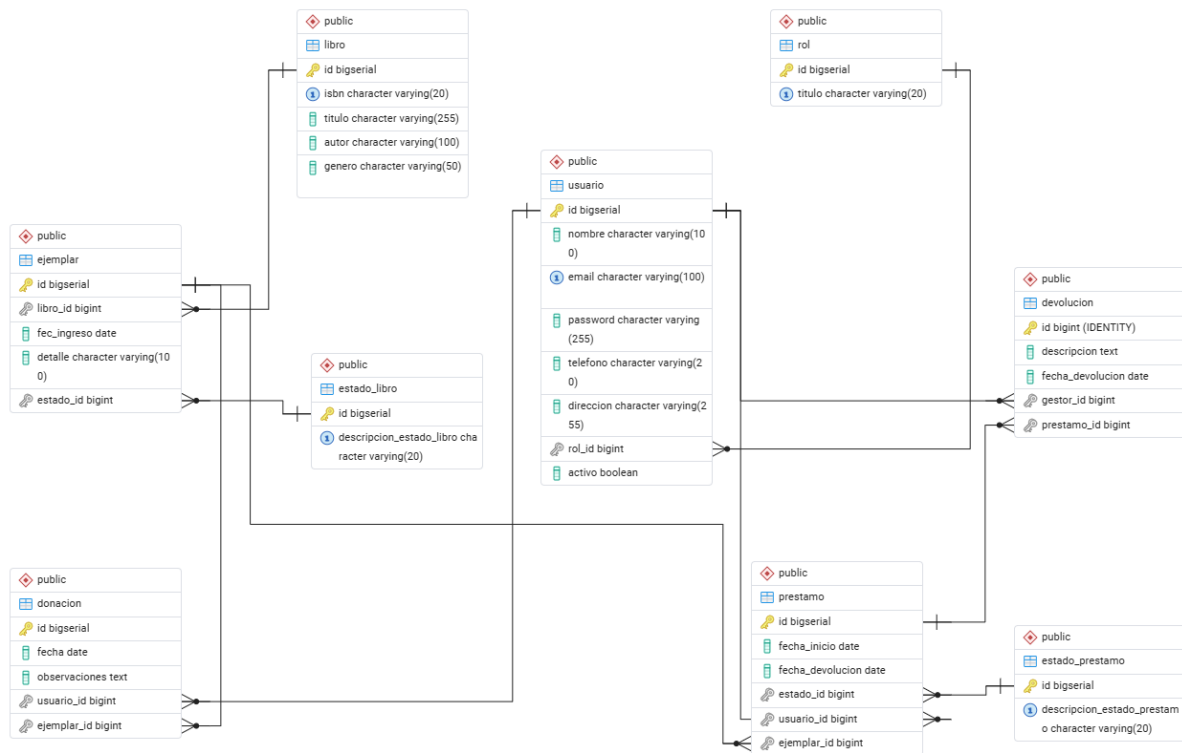
5.2 Análisis del sistema

5.3 Diseño del sistema:

5.3.1 Diseño de la base de datos

- rol: Gestiona los diferentes perfiles o niveles de acceso.
- estado_libro: Define las condiciones en las que puede estar un libro (ej. disponible, prestado, dañado).
- estado_prestamo: Cataloga el estado del trámite de un préstamo (ej. activo, devuelto, atrasado).
- usuario: Almacena a los lectores y gestores (incluye nombre, email, password, etc.) y su rol asignado.
- libro: Guarda la metadata universal de una obra (ISBN, título, autor, género).
- ejemplar: Representa el libro físico particular en la biblioteca (se relaciona con libro e indica su estado_libro).
- prestamo: Registra los movimientos de salida de un ejemplar hacia un usuario, con fechas y su respectivo estado.
- donacion: Registra las donaciones realizadas por los usuarios (relacionado con el nuevo ejemplar donado).
- devolucion: Tabla encargada de registrar el retorno de los préstamos, indicando el gestor que procesa la devolución y descripciones adicionales.

El diagrama entidad relación podemos verlo a continuación.



Configuración

```

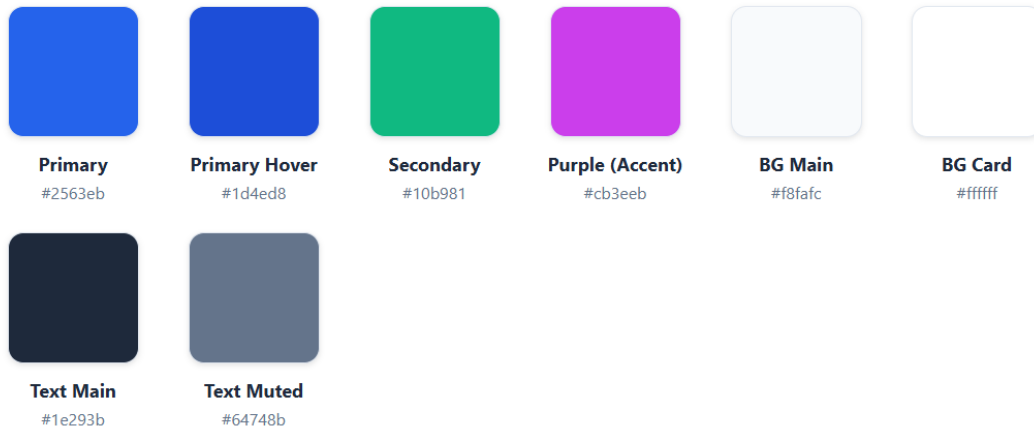
1 spring.application.name=libraryPro
2
3 # Configuración de Base de Datos (PostgreSQL 17)
4 spring.datasource.url=jdbc:postgresql://localhost:5432/libraryPro_db
5 spring.datasource.username=postgres
6 spring.datasource.password=admin
7 spring.datasource.driver-class-name=org.postgresql.Driver
8
9 # JPA / Hibernate
10 spring.jpa.hibernate.ddl-auto=update
11 spring.jpa.show-sql=true
12 spring.jpa.properties.hibernate.format_sql=true
13 spring.jpa.database-platform=org.hibernate.dialect.PostgreSQLDialect
14
15 # Puerto del servidor
16 server.port=8087
17
  
```

5.3.2 Diseño de la interfaz de usuario

Este documento presenta un desglose de la paleta de colores, la tipografía y los estilos de los componentes principales utilizados en el sistema **LibraryPro**, basado en el archivo de estilos globales.

Paleta de Colores

El sistema utiliza una paleta moderna, con tonos fríos como principales y un verde esmeralda para destacar elementos secundarios.



Tipología

Se utiliza la fuente 'Inter' como tipografía principal. Esta fuente está optimizada para interfaces de usuario, ofreciendo excelente legibilidad en diferentes tamaños de pantalla.

Pila completa: 'Inter', system-ui, -apple-system, sans-serif

Estilo de componentes

Tarjetas (Cards)

Utilizan un fondo blanco puro (`var(--bg-card)`), con bordes redondeados pronunciados de `1rem` y una sombra sutil para dar profundidad. Cuentan con un borde tenue (`#e2e8f0`) para definir las mejor en fondos claros.

Botones (Buttons)

Los botones tienen bordes redondeados (`0.5rem`) y no utilizan bordes (`border: none`). Cuentan con transiciones suaves (`0.2s`) al pasar el cursor (hover). El botón primario usa el color azul (`#2563eb`) y se oscurece al interactuar.

Barras de Navegación (Sidebar & Nav Links)

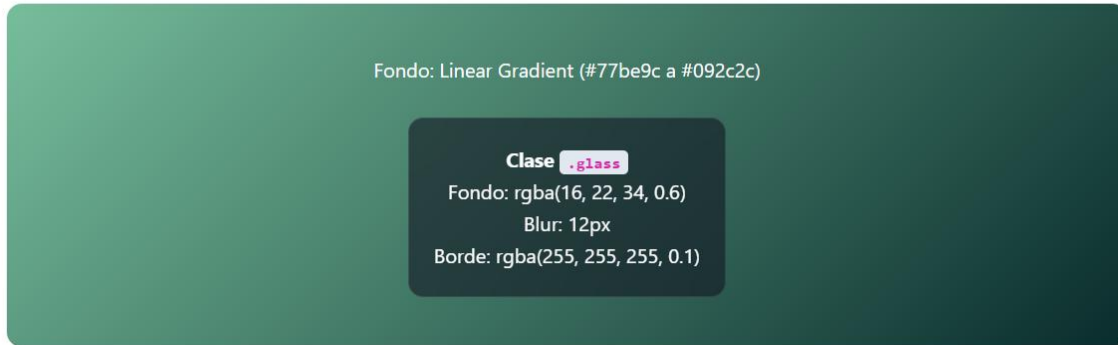
Los enlaces de navegación usan colores apagados que se iluminan al hacer hover, aplicando un sutil fondo blanco translúcido (`rgba(255, 255, 255, 0.1)`) para indicar interactividad.

Scrollbars Personalizados

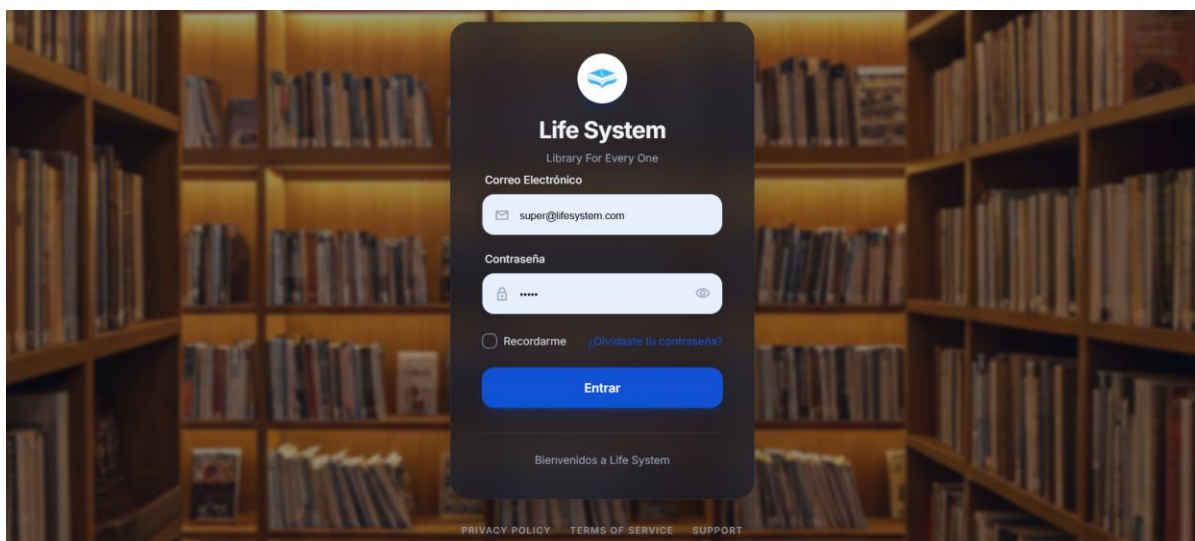
Para los contenedores con estilo "Glass", se ocultan los scrollbars del sistema operativo para mantener la ilusión de interfaz inmersiva, reemplazándolos con barras de desplazamiento sutiles y semitransparentes.

✨ Efecto Glassmorphism (Cristal)

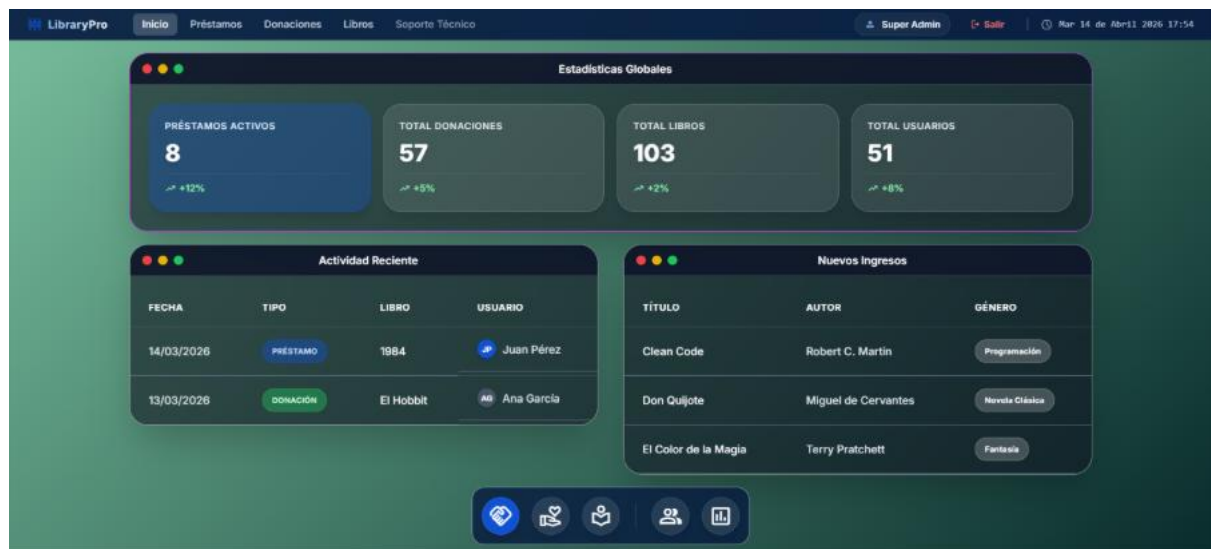
Una característica destacada del diseño es el uso del "Glassmorphism", especialmente sobre el fondo estilo escritorio (.desktop-bg). Combina fondos oscuros semitransparentes con desenfoque de fondo (backdrop-filter).



Login



Inicio



Prestamos

LibraryPro Inicio **Préstamos** Donaciones Libros Soporte Técnico Super Admin Salir Mar 14 de Abr 11 2026 17:54

Registrar Préstamo

Crear Perfil

BUSCAR LECTOR

Nombre, teléfono o correo

LIBRO A PRESTAR (EJEMPLAR)

Busque o seleccione un ejemplar...

FECHA DE INICIO **DEVOLUCIÓN (ESTIMADA)**

Últimos Préstamos

FECHA	LIBRO	LECTOR
03/04/2026	Programacion POO de Java 2026	Juan D. Melénd...
01/04/2026	Libro Genérico Vol 99	Juan D. Melénd...
23/03/2026	Un Mundo Feliz	Super Admin
20/01/2024	Libro Genérico Vol 59	Raúl Romero
19/03/2024	Libro Genérico Vol 58	Rubén Fernánd...
26/03/2024	Libro Genérico Vol 57	Laura López
18/03/2024	Libro Genérico Vol 56	Beatriz Ortiz
03/02/2024	Libro Genérico Vol 55	Miguel Vázquez
08/02/2024	Libro Genérico Vol 54	Beatriz Lozano

-> crear perfil

LibraryPro Inicio Préstamos Donaciones Libros Soporte Técnico Super Admin Salir Mar 14 de Abr 11 2026 17:56

Crear Nuevo Perfil Lector

NOMBRE COMPLETO *

Ej. Juan Pérez

CORREO ELECTRÓNICO * TELÉFONO *

correo@ejemplo.com Número de contacto

DIRECCIÓN (OPCIONAL)

Dirección de residencia

CONTRASEÑA PARA ACCESO (OPCIONAL)

Generar una contraseña

Guardar Perfil Cancelar

Donaciones

LibraryPro Inicio Préstamos Donaciones Libros Soporte Técnico Super Admin Salir Mar 14 de Abr 11 2026 17:57

Registrar Donación

INFORMACIÓN DEL LIBRO

Buscar libro por título... Buscar + Crear Libro

DATOS DEL DONANTE

Buscar por nombre... Buscar + Perfil Nuevo

FECHA CONDICIÓN OBSERVACIONES

14/04/2026 Nuevo Detalles adicionales...

Registrar Donación Cancelar

Últimas Donaciones

FECHA	LIBRO / AUTOR	DONANTE
03/04/2026	Programacion de Objetos con Php 8 Autor Desconocido	Juan D. Meléndez...
02/04/2026	Programacion de Objetos con Php 8 Autor Desconocido	Juan D. Meléndez...
01/04/2026	Programacion POO de Java 2026 Juan Rullo	Juan Morales
21/03/2026	El Principito Antoine de Saint-Exupéry	Super Admin
21/03/2026	El Principito Antoine de Saint-Exupéry	Super Admin
21/03/2026	El Principito Antoine de Saint-Exupéry	Super Admin
21/03/2026	El Principito Antoine de Saint-Exupéry	Super Admin

Libros

LibraryPro Inicio Préstamos Donaciones Libros Soporte Técnico Super Admin Salir Mar 14 de Abr 11 2026 17:57

Gestión de Libros

TÍTULO AUTOR GÉNERO

Buscar por título... Buscar por autor... Buscar por género... Buscar Limpiar + Nuevo Libro

ID	TÍTULO	AUTOR	GÉNERO	ESTADO	ACCIONES
1	100 años de soledad	Gabriel Garcia Márquez	REALISMO MÁGICO	DISPONIBLE	Editar
2	Don Quijote de la Mancha	Miguel de Cervantes	CLÁSICO	DISPONIBLE	Editar
3	1984	George Orwell	DISTOPIA	DISPONIBLE	Editar
4	El Principito	Antoine de Saint-Exupéry	INFANTE	DISPONIBLE	Editar
5	El Hobbit	J.R.R. Tolkien	FANTASÍA	DISPONIBLE	Editar
6	Orgullo y Prejuicio	Jane Austen	ROMANCE	DISPONIBLE	Editar
7	El Señor de los Anillos	J.R.R. Tolkien	FANTASÍA	DISPONIBLE	Editar

Soporte Tecnico

LibraryPro Inicio Préstamos Donaciones Libros **Soporte Técnico** Super Admin Salir Mar 14 de Abr 11 2026 17:58

Acerca del Proyecto

Detalles técnicos e información de contacto

DESARROLLADOR / AUTOR
Juan D. Meléndez Peña
 juanmelendez.pe@gmail.com | juand.mp@outlook.es

TÍTULO DEL PROYECTO
 Proyecto Final de Técnico Superior de Desarrollo de Aplicaciones Web - España 2026
 LibraryPro - Sistema de Gestión de Biblioteca

FRONTEND & UI
 Angular 21
 Tailwind CSS

BACKEND & DATABASE
 Java 21 (Spring Framework 3.4)
 PostgreSQL 17

INSTITUCIÓN
 IES Ribera de Castilla - Valladolid, España

Lectores

LibraryPro Inicio Préstamos Donaciones Libros **Soporte Técnico** Super Admin Salir Mar 14 de Abr 11 2026 18:00

Lectores GESTORES

Buscar lector por nombre o email... + NUEVO LECTOR

PERFIL DE USUARIO	CONTACTO / CORREO	ROL DE ACCESO	ESTADO ACTUAL	PANEL DE ACCIONES
Juan D. Meléndez Peña 167410321	juanmelendez.pe@gmail.com	LECTOR	ACTIVO	Desactivar Editar
Beatriz Ortiz 167410321	beatriz.ortiz4@ejemplo.com	LECTOR	ACTIVO	Desactivar Editar
Raquel Ortiz 167410321	raquel.ortiz4@ejemplo.com	LECTOR	ACTIVO	Desactivar Editar
Rubén Pérez 167410321	ruben.perez4@ejemplo.com	LECTOR	ACTIVO	Desactivar Editar

Gestores

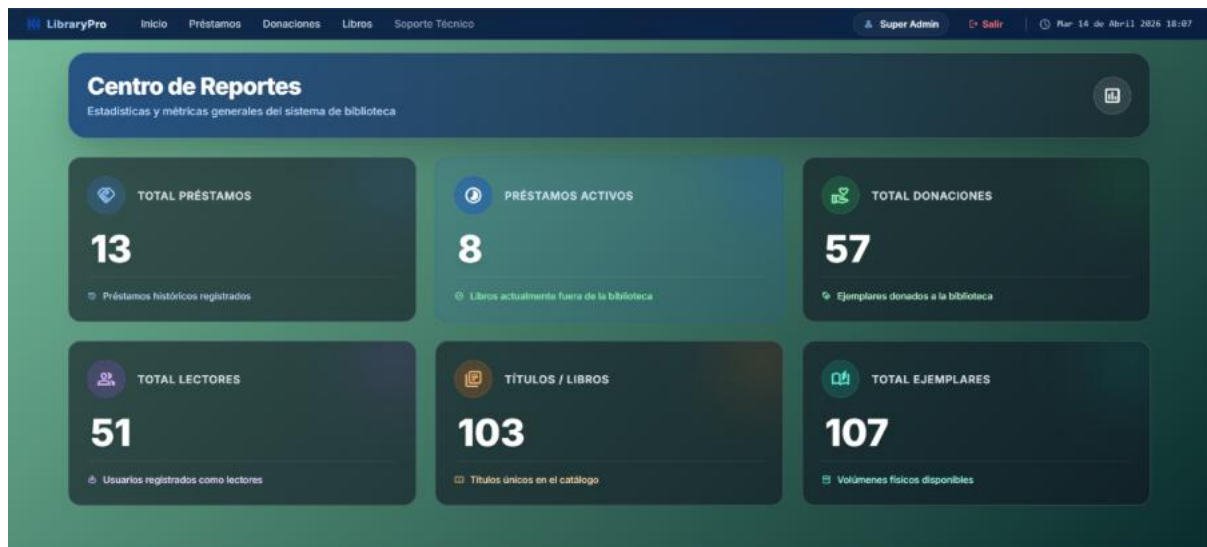
LibraryPro Inicio Préstamos Donaciones Libros **Soporte Técnico** Super Admin Salir Mar 14 de Abr 11 2026 18:01

Lectores **Gestores**

Buscar gestor por nombre o email... + NUEVO GESTOR

PERFIL DE USUARIO	CONTACTO / CORREO	ROL DE ACCESO	ESTADO ACTUAL	PANEL DE ACCIONES
John Sunday 167410321	jsunday@gmail.com	GESTOR	ACTIVO	Desactivar Editar
Karina Nieto 131430789	karina@ejemplo.com	ADMIN	ACTIVO	Desactivar Editar
Juan D. Meléndez Peña 167410321	juanmelendez.pe@ssdk.com	ADMIN	ACTIVO	Desactivar Editar
Super Admin 800009803	super@librarypro.com	SUPERADMIN	ACTIVO	Desactivar Editar

Reportes



Actividades – Usuarios

Prestamos

BE Beatriz Ortiz
beatriz.ortiz49@ejemplo.com • 690076407

[← Volver](#)

PRÉSTAMOS (1) | DEVOLUCIONES (1) | DONACIONES (1)

LIBRO	FECHA INICIO/LÍMITE	ESTADO	ACCIÓN
Libro Genérico Vol 56 ID Ejemplar: 32	3/18/24 4/8/26	DEVUELTO	

Devoluciones

LibraryPro Inicio Préstamos Donaciones Libros Soporte Técnico Super Admin Salir Mar 14 de Abr 11 2026 18:19

BE Beatriz Ortiz
beatriz.ortiz49@ejemplo.com ~ 690078407 [Volver](#)

PRESTAMOS (1) **DEVOLUCIONES (1)** DONACIONES (1)

LIBRO DEVUELTO	FECHA RETORNO	GESTOR RECEPTOR	DESCRIPCIÓN
Libro Genérico Vol 56	4/6/26, 12:00 AM	Ana Administradora	devolucion conforme

[Inicio](#) [Préstamos](#) [Donaciones](#) [Libros](#) [Soporte Técnico](#)

Donaciones

LibraryPro Inicio Préstamos Donaciones Libros Soporte Técnico Super Admin Salir Mar 14 de Abr 11 2026 18:28

BE Beatriz Ortiz
beatriz.ortiz49@ejemplo.com ~ 690078407 [Volver](#)

PRESTAMOS (1) DEVOLUCIONES (1) **DONACIONES (1)**

LIBRO DONADO	FECHA	OBSERVACIONES
Los Detectives Salvajes	2/21/23	Donado por usuario

[Inicio](#) [Préstamos](#) [Donaciones](#) [Libros](#) [Soporte Técnico](#)

Usuario Lector – editar

EDITAR LECTOR

DATOS PERSONALES

NOMBRE COMPLETO *
Juan D. Meléndez Peña

CORREO ELECTRÓNICO *
juanmelendez.pe@gmail.com

TELÉFONO
987456321

DIRECCIÓN
calle hornos caleros 37

ACCESO Y SISTEMA

☒ **PERFIL ACTIVO**

Cancelar **Guardar Cambios**

Usuario Gestor - Editar

EDITAR GESTOR/ADMIN

DATOS PERSONALES

NOMBRE COMPLETO *
Karina Nieto

CORREO ELECTRÓNICO *
karina@ejemplo.com

TELÉFONO
123456789

DIRECCIÓN
calle hornos caleros 40

ACCESO Y SISTEMA

CONTRASEÑA
.....

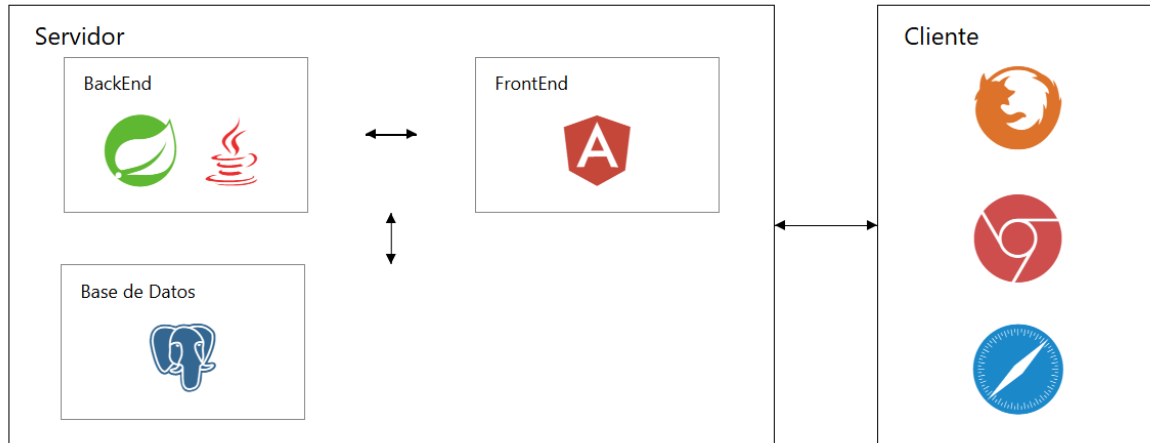
ROL DEL SISTEMA
Administrador

☒ **PERFIL ACTIVO**

Cancelar **Guardar Cambios**

5.3.3 Diseño de la aplicación

La aplicación tiene el siguiente diseño a la hora de comunicarse entre servidor y cliente.



5.4 Implementación:

5.4.1 Entorno de desarrollo

- Backend -> Eclipse IDE
- Frontend -> VS Code - Antigravity

5.4.2 Estructura del código

1. Estructura del Backend (AppBack/libraryPro/)

El backend está construido con **Java** usando el framework **Spring Boot**. Utiliza Maven como gestor de dependencias y sigue el patrón de diseño clásico de capas (MVC/API REST).

La estructura interna de paquetes (ubicada en `src/main/java/com/ssdk/libraryPro/`) es la siguiente:

- **model/ (Capa de Datos/Entidades):** Aquí se encuentran las clases Java que representan las tablas de la base de datos PostgreSQL (ej. `Usuario.java`, `Libro.java`, `Prestamo.java`). Estas clases utilizan anotaciones JPA/Hibernate (`@Entity`, `@Table`) para mapear los objetos a la base de datos.
- **repository/ (Capa de Acceso a Datos):** Contiene las interfaces (ej. `UsuarioRepository`, `LibroRepository`) que heredan de `JpaRepository`. Spring Boot usa esto para autogenerar todas las consultas SQL (CRUD básico) sin necesidad de escribir código manual.
- **service/ (Capa de Lógica de Negocio):** Aquí reside la "inteligencia" de la aplicación. Se procesan validaciones, cálculos y la lógica antes de guardar o responder datos (por ejemplo, verificar si un ejemplar está disponible antes de procesar un préstamo).

- **controller/ (Capa de Presentación / API REST):** Es el punto de entrada de las peticiones HTTP del Frontend. Aquí se definen los Endpoints (@RestController, @GetMapping, @PostMapping). Los controladores reciben la petición de Angular, llaman al service correspondiente y devuelven la respuesta en formato JSON.
 - **AppLibraryPro.java:** Es la clase principal o punto de arranque que contiene el método main y levanta el servidor Tomcat embebido.
-

2. Estructura del Frontend (AppFronD/src/app/)

El frontend está desarrollado con **Angular** y emplea fuertemente TypeScript. Su estructura está modularizada y pensada para escalabilidad.

- **components/ (Vistas e Interfaces):** Agrupa todos los elementos visuales de la aplicación. En lugar de estar todos mezclados, están perfectamente categorizados por módulo funcional:
 - /login: Pantalla de inicio de sesión.
 - /inicio: El Dashboard principal o Home.
 - /libros y /usuarios: Gestión de catálogo y responsables.
 - /prestamos, /devoluciones, /donaciones: Formularios transaccionales para los flujos operativos de la biblioteca.
 - /reportes y /soporte: Vistas analíticas e informativas.
- **services/ (Conexión con el Backend):** Contiene archivos como auth.service.ts o libro.service.ts. Su única responsabilidad es usar el HttpClient de Angular para hacer las peticiones GET, POST, PUT hacia los endpoints del Backend de Spring Boot.
- **models/ (Tipado Estricto):** Aquí se definen las interfaces TypeScript (models.ts) que replican la estructura de los objetos del backend. Esto asegura que Angular sepa exactamente qué campos tiene un Libro o un Usuario, evitando errores de escritura.
- **guards/ (Seguridad de Rutas):** Aloja los Guardianes (ej. auth.guard.ts). Estos scripts se ejecutan antes de entrar a un componente y verifican si el usuario tiene permiso o está logueado para ver la pantalla.
- **Archivos Raíz (app.routes.ts, app.component.ts):** Orquestan toda la aplicación. app.routes.ts define qué componente se carga en cada URL, mientras que el componente raíz provee el esqueleto general (menús, barras de navegación, contenedores).

💡 Resumen del Flujo

1. El usuario interactúa con un componente en el **Frontend** (ej. botón "Registrar Préstamo").
2. El componente llama a un método en la carpeta services/ de Angular.
3. El servicio hace una petición HTTP al controller/ del **Backend**.
4. El controlador delega la tarea al service/ de Java (donde se valida la lógica de negocio).
5. El servicio de Java utiliza el repository/ para interactuar con la Base de Datos PostgreSQL y retorna la confirmación hasta llegar de nuevo a la pantalla del usuario.

5.5 Pruebas

Informe de Pruebas Unitarias (Backend)

Proyecto: LiFe System

Capa Analizada: Lógica de Negocio (`UsuarioService.java`)

Tecnologías: Java 21, Spring Boot, JUnit 5, Mockito

3

Total de Pruebas

3

Pruebas Exitosas

100%

Tasa de Aprobación

Detalle de Ejecución

Nombre del Test	Objetivo Validado	Estado
<code>testCrearUsuario_GuardarExitoso</code>	Simula el registro de un nuevo lector/gestor. Valida que el objeto se guarde utilizando el repositorio correctamente y retorna los valores esperados.	PASSED
<code>testLogin_BuscarPorEmail_Exitoso</code>	Simula un Login exitoso. Valida la búsqueda de un email registrado en la BD y verifica que la contraseña mockeada coincida para la autorización.	PASSED
<code>testLogin_BuscarPorEmail_FalloNoExiste</code>	Simula un Login fallido. Valida el comportamiento del sistema cuando se ingresa un correo que no está registrado en la base de datos (Retorno vacío).	PASSED

Conclusión Técnica

Las pruebas unitarias sobre el módulo de usuarios han finalizado de manera satisfactoria. Gracias a la utilización de **Mockito**, se logró aislar la capa de servicio (UsuarioService) sin necesidad de conectar a una base de datos real (PostgreSQL).

Esto garantiza que la lógica central del negocio —la creación de perfiles y la verificación de credenciales básicas para el Login— es robusta y responde exactamente como se espera ante escenarios de éxito y escenarios de error.

```

▼ > src/test/java
  > > com.ssd.libraryPro
  ▼ > com.ssd.libraryPro.service
    > > UsuarioServiceTest.java

```

```

1 package com.ssd.libraryPro.service;
2
3 import com.ssd.libraryPro.model.Rol;
4
18
19 @ExtendWith(MockitoExtension.class)
20 public class UsuarioServiceTest {
21
22     @Mock
23     private UsuarioRepository usuarioRepository;
24
25     @InjectMocks
26     private UsuarioService usuarioService;
27
28     private Usuario usuarioMock;
29
30     @BeforeEach
31     void setUp() {
32         // Crear un Rol simulado para el usuario
33         Rol rol = new Rol(1L, "lector");
34
35         // Construir un Usuario simulado utilizando el constructor completo
36         usuarioMock = new Usuario(
37             1L,
38             "Juan Perez",
39             "juan@correo.com",
40             "password123",
41             "123456789",
42             "Calle Falsa 123",
43             rol,
44             true
45         );
46     }

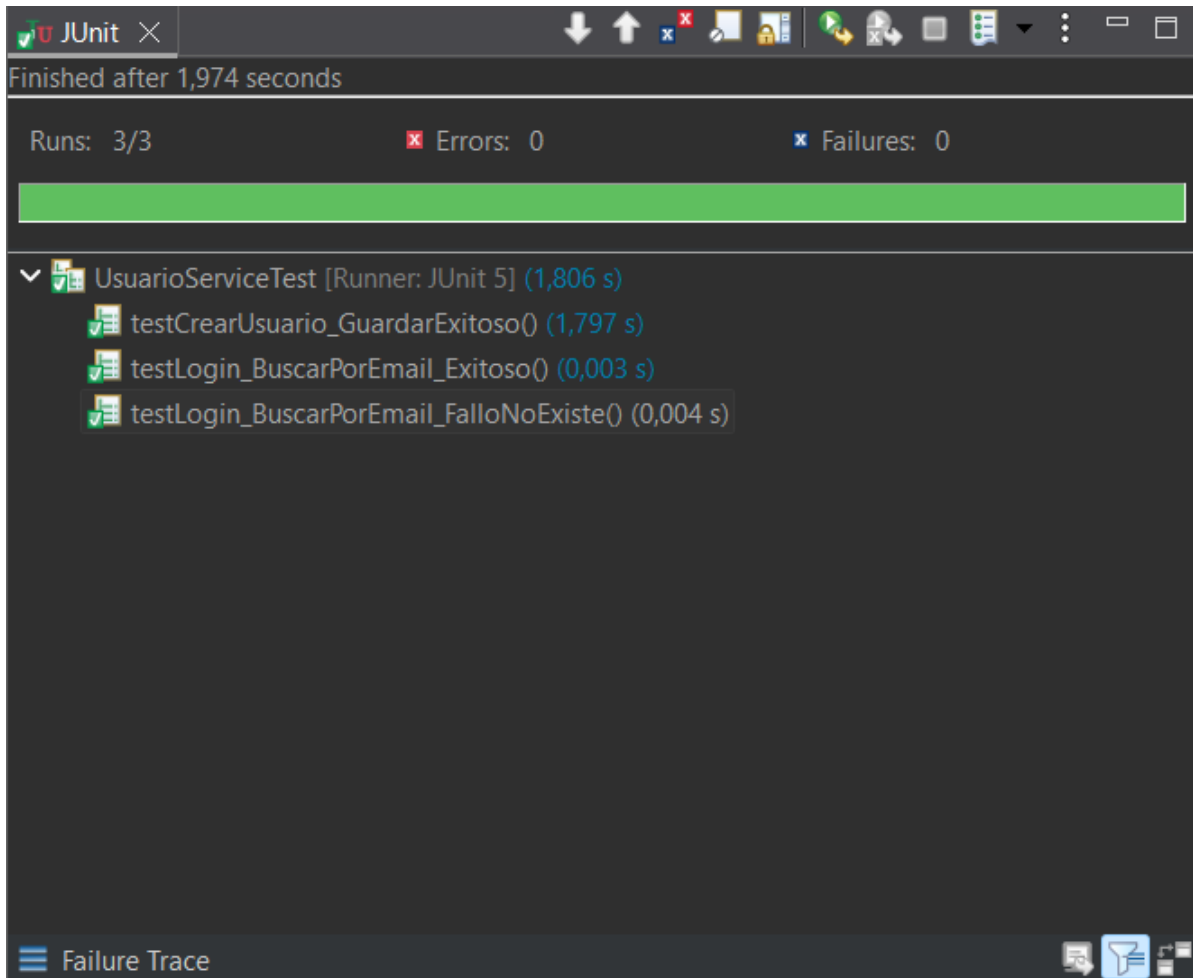
```

```

48 @Test
49 void testCrearUsuario_GuardarExitoso() {
50     // Arrange: Instruimos al mock del repositorio para que retorne usuarioMock
51     // cuando se intente guardar cualquier objeto de tipo Usuario.
52     when(usuarioRepository.save(any(Usuario.class))).thenReturn(usuarioMock);
53
54     // Act: Ejecutamos el método del servicio encargado de crear el usuario.
55     Usuario nuevoUsuario = new Usuario();
56     nuevoUsuario.setNombre("Juan Perez");
57     nuevoUsuario.setEmail("juan@correo.com");
58
59     Usuario usuarioGuardado = usuarioService.guardar(nuevoUsuario);
60
61     // Assert: Validamos que el resultado no sea nulo y contenga los datos esperados.
62     assertNotNull(usuarioGuardado, "El usuario guardado no debe ser nulo");
63     assertEquals("Juan Perez", usuarioGuardado.getNombre(), "El nombre debe coincidir con el mock");
64     assertEquals("juan@correo.com", usuarioGuardado.getEmail(), "El correo debe coincidir con el mock");
65     assertEquals(1L, usuarioGuardado.getId(), "El ID generado debe ser 1");
66
67     // Verificamos que el repositorio efectivamente llamó al método save() exactamente una vez.
68     verify(usuarioRepository, times(1)).save(nuevoUsuario);
69 }
70
71 @Test
72 void testLogin_BuscarPorEmail_Exitoso() {
73     // Arrange: En este backend la base del login (comprobación de credenciales)
74     // recae en buscar el usuario por su email. Simulamos que si existe en la BD.
75     when(usuarioRepository.findByEmail("juan@correo.com")).thenReturn(Optional.of(usuarioMock));
76
77     // Act: Solicitamos el usuario al servicio
78     Optional<Usuario> usuarioEncontrado = usuarioService.buscarPorEmail("juan@correo.com");
79
80     // Assert: Comprobamos que el usuario fue encontrado y tiene la contraseña correcta (Login Ok)
81     assertTrue(usuarioEncontrado.isPresent(), "El usuario debería existir en la base de datos");
82     assertEquals("password123", usuarioEncontrado.get().getContraseña(), "La contraseña debe coincidir para proceder con la autorización");
83
84     // Verificamos que el método del repositorio se ejecutó una vez
85     verify(usuarioRepository, times(1)).findByEmail("juan@correo.com");
86 }

```

```
87
88     @Test
89     void testLogin_BuscarPorEmail_FalloNoExiste() {
90         // Arrange: Simulamos un intento de login con un correo que no está registrado
91         when(usuarioRepository.findByEmail("desconocido@correo.com")).thenReturn(Optional.empty());
92
93         // Act: Intentamos buscar
94         Optional<Usuario> usuarioEncontrado = usuarioService.buscarPorEmail("desconocido@correo.com");
95
96         // Assert: Validamos que retorne vacío (Login Fallido)
97         assertFalse(usuarioEncontrado.isPresent(), "No debería encontrar ningún usuario con ese correo");
98
99         // Verificamos que se ejecutó la consulta en el repositorio
100        verify(usuarioRepository, times(1)).findByEmail("desconocido@correo.com");
101    }
102 }
103
```



JUnit

Finished after 1,974 seconds

Runs: 3/3 ✖ Errors: 0 ✖ Failures: 0

▼  UsuarioServiceTest [Runner: JUnit 5] (1,806 s)

-  testCrearUsuario_GuardarExitoso() (1,797 s)
-  testLogin_BuscarPorEmail_Exitoso() (0,003 s)
-  testLogin_BuscarPorEmail_FalloNoExiste() (0,004 s)

Failure Trace

6 Manuales de usuario

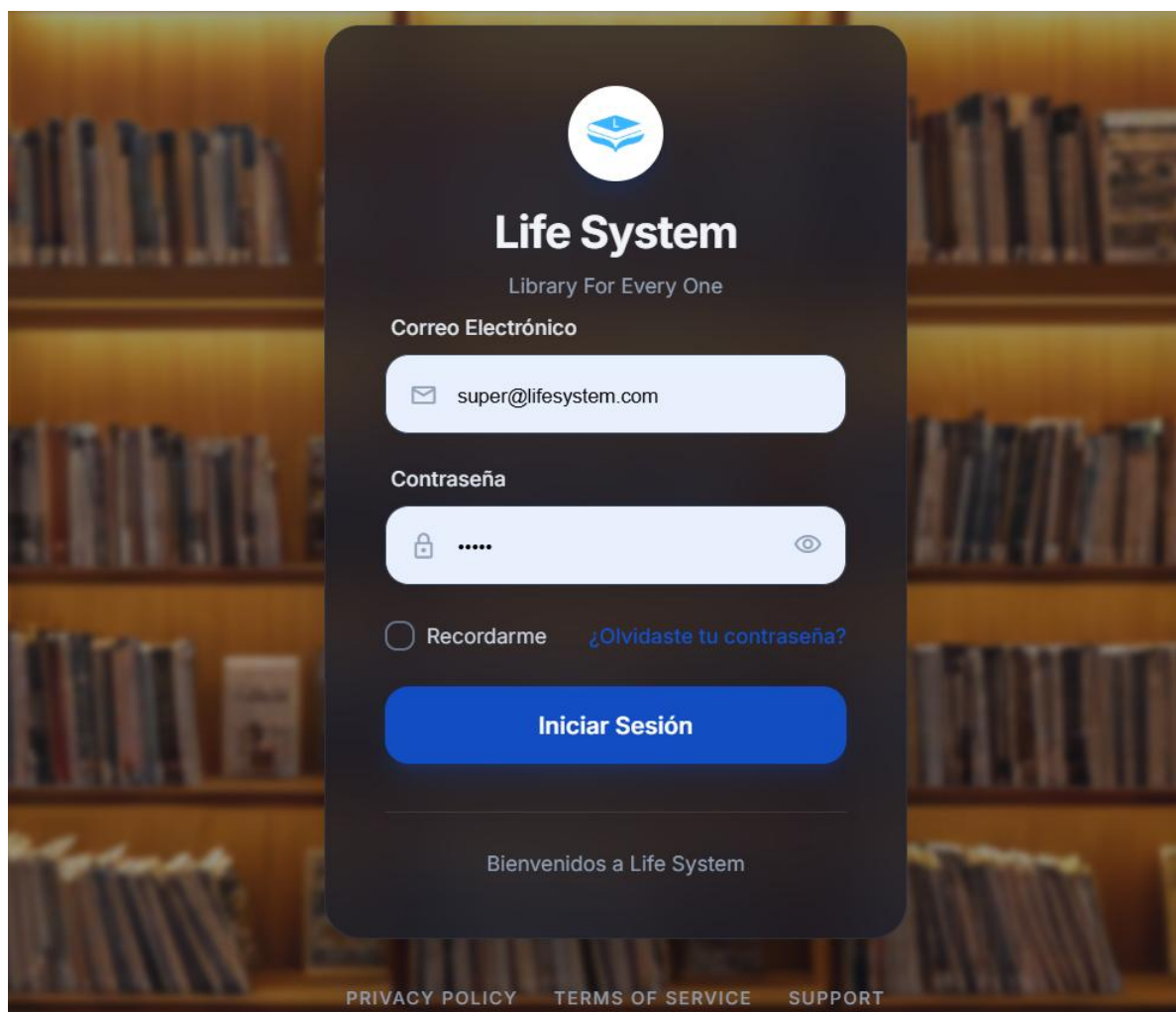
6.1 Manual de usuario

Guía de capacitación completa para la gestión bibliotecaria en LiFe System.

1. Inicio de Sesión

Para acceder a las funcionalidades del sistema, es obligatorio autenticarse con credenciales válidas.

1. Abra el navegador web e ingrese a la dirección del sistema (ej. <http://localhost:4200>).
2. Visualizará la pantalla principal de **Login**.
3. Ingrese su **Correo Electrónico** registrado y su **Contraseña**.
4. Haga clic en el botón "**Ingresar**" o "**Iniciar Sesión**".



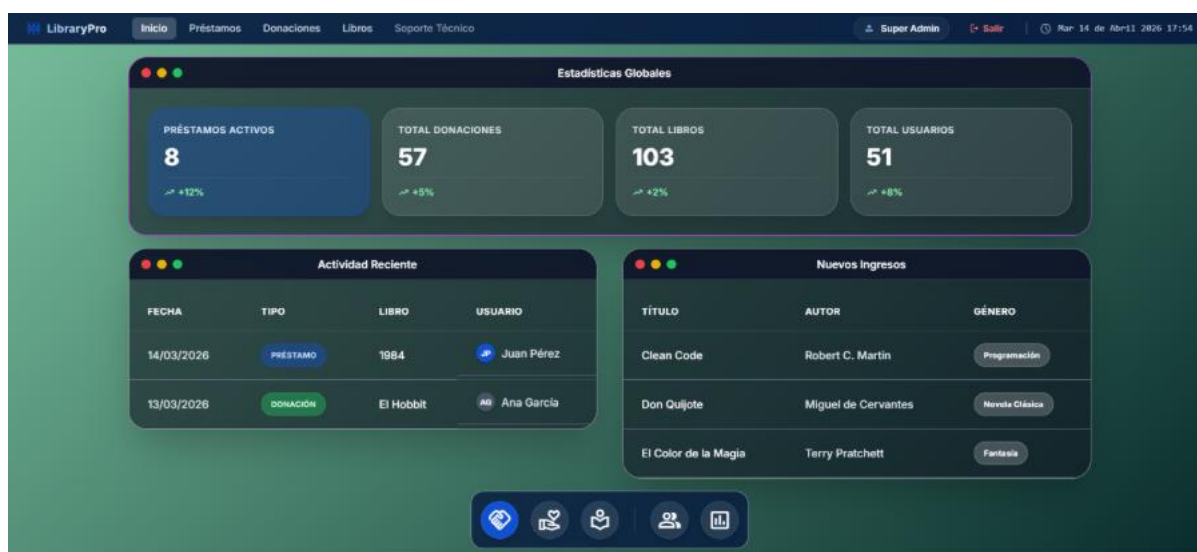
Usuario: super@lifesystem.com

Contraseña: super

2. Dashboard Principal (Home)

Una vez autenticado, ingresará al Dashboard principal. Este espacio sirve como panel de control rápido.

1. En la pantalla de inicio, encontrará accesos rápidos a las funciones más utilizadas.
2. Dependiendo de su nivel de acceso (Lector o Gestor), visualizará notificaciones, préstamos pendientes o estadísticas resumidas.
3. Utilice el menú de navegación (lateral o inferior) para desplazarse entre los diferentes módulos del sistema.



3. Catálogo de Libros

Módulo destinado a la búsqueda y gestión del inventario literario de la biblioteca.

Buscador de Libros

1. Acceda al módulo **"Libros"**.
2. Utilice la barra de búsqueda para filtrar obras por Título, Autor, Género o ISBN.
3. Haga clic sobre un libro en la lista para ver su detalle y la cantidad de ejemplares disponibles.

Agregar un Libro Nuevo

1. Dentro de la vista del catálogo, haga clic en **"Nuevo Libro"**.
2. Complete el formulario con el ISBN, Título, Autor (selección autocompletada) y Género.
3. Guarde los cambios. Esto creará la "ficha" universal del libro en la base de datos.

Filtrar por titulo

The screenshot shows the 'Gestión de Libros' (Book Management) interface. At the top, there's a navigation bar with 'Inicio', 'Préstamos', 'Donaciones', 'Libros', and 'Soporte Técnico'. The 'Libros' tab is active. Below the navigation bar, there's a search section with three input fields: 'TÍTULO' (containing 'principito'), 'AUTOR' (with a placeholder 'Buscar por autor...'), and 'GÉNERO' (with a placeholder 'Buscar por género...'). There are buttons for 'Buscar' (Search), 'Limpiar' (Clear), and '+ Nuevo Libro' (Add New Book). Below the search section is a table with the following data:

ID	TÍTULO	AUTOR	GÉNERO	ESTADO	ACCIONES
4	El Principito	Antoine de Saint-Exupéry	INFANTIL	DISPONIBLE	Editar
101	El Principito	Antoine de Saint-Exupéry	INFANTIL	DISPONIBLE	Editar
102	El Principito	Antoine de Saint-Exupéry	INFANTIL	DISPONIBLE	Editar
103	El Principito	Antoine de Saint-Exupéry	INFANTIL	DISPONIBLE	Editar
104	El Principito	Antoine de Saint-Exupéry	INFANTIL	DISPONIBLE	Editar

Crear nuevo libro

The screenshot shows the 'Añadir Nuevo Libro al Catálogo' (Add New Book to Catalog) form. The form has the following fields and values:

- TÍTULO DEL LIBRO ***: Programación Orientados a Objetos con Java 25
- AUTOR ***: Juan MP
- GÉNERO PRINCIPAL**: tecnologia
- ISBN (OPCIONAL)**: 987654321321

At the bottom of the form are two buttons: 'Guardar Libro' (Save Book) and 'Cancelar' (Cancel). Below the form, there's a navigation bar with icons for 'Inicio', 'Préstamos', 'Donaciones', 'Libros', and 'Soporte Técnico'.

Notificación indicando que se ha creado el libro en la base de datos.

The screenshot shows the 'Añadir Nuevo Libro al Catálogo' form with a success notification. The notification text is: 'Libro registrado exitosamente en el catálogo. Puedes registrar ejemplares físicos a través de una Donación o Ingreso.' (Book registered successfully in the catalog. You can register physical copies through a Donation or Income.) There is an 'Aceptar' (Accept) button next to the notification. The form fields and values are the same as in the previous screenshot.

4. Gestión de Usuarios (Responsables)

Panel centralizado para administrar a todas las personas que interactúan con la biblioteca.

Crear o Editar un Usuario (Lector / Gestor)

1. Seleccione el módulo **"Gestionar Responsables"**.
2. Navegue entre las pestañas "Lectores" y "Gestores" según el perfil que desee administrar.
3. Haga clic en **"Nuevo"** para registrar a alguien, o use el botón de edición junto a un usuario existente.
4. En el modal, modifique los datos personales y de contacto. Guarde los cambios.

Dashboard de Actividad del Usuario

1. Al visualizar la lista de usuarios, seleccione el botón de **"Ver Actividad"** de un lector específico.
2. Esto abrirá un panel con el historial interactivo de dicho usuario: Préstamos activos, devoluciones pasadas y donaciones realizadas.

1. Modulo de lectores

LECTORES

GESTORES

Buscar lector por nombre o email...

+ NUEVO LECTOR

Directorio de Lectores

PERFIL DE USUARIO	CONTACTO / CORREO	ROL DE ACCESO	ESTADO ACTUAL	PANEL DE ACCIONES
JU Juan D. Meléndez Peña 987456321	✉ juanmelendez.pe@gmail.com	LECTOR	● ACTIVO	Desactivar Editar
BE Beatriz Ortiz 690078407	✉ beatriz.ortiz49@ejemplo.com	LECTOR	● ACTIVO	Desactivar Editar
RA Raquel Ortiz 617841015	✉ raquel.ortiz48@ejemplo.com	LECTOR	● ACTIVO	Desactivar Editar
RU Rubén Pérez 639482968	✉ rubén.pérez47@ejemplo.com	LECTOR	● ACTIVO	Desactivar Editar

2. Modulo Gestores

DIRECTORIO DE ADMINISTRADORES				
PERFIL DE USUARIO	CONTACTO / CORREO	ROL DE ACCESO	ESTADO ACTUAL	PANEL DE ACCIONES
 john sunday 987456321	 Jsunday@gmail.com	GESTOR 	● ACTIVO	 Desactivar  Editar
 Karina Nieto 123456789	 karina@ejemplo.com	ADMIN 	● ACTIVO	 Desactivar  Editar
 Juan D. Meléndez Peña 987456321	 juanmelendez.pe@ssdk.com	ADMIN 	● ACTIVO	 Desactivar  Editar
 Super Admin 600000002	 super@lifesystem.com	SUPERADMIN 	● ACTIVO	 Desactivar  Editar

Creación de nuevo Usuario

TREM

Recibidos - juan...

1340] [...]

Jira

haciones

Libros

Sopor

Super Admin

localhost:4200 dice

Usuario creado, presione para volver a préstamos.

Aceptar

NOMBRE COMPLETO *

Patricia Power

CORREO ELECTRÓNICO *

patriciapower@ejemplo.com

TELÉFONO *

123456789

DIRECCIÓN (OPCIONAL)

calle real 1

CONTRASEÑA PARA ACCESO (OPCIONAL)

test

Guardar Perfil

Cancelar

Se verifica que el nuevo usuario lector se ha creado correctamente

The screenshot shows the 'LiFe System' interface. The top navigation bar includes 'Inicio', 'Préstamos', 'Donaciones', 'Libros', and 'Soporte Técnico'. The main content area is divided into two panels. The left panel, titled 'Registrar Préstamo', contains a form for creating a new loan. It has a 'Crear Perfil' section, a 'BUSCAR LECTOR' section with a search bar containing 'Patricia Power' and a 'Buscar' button, and a 'LECTOR SELECCIONADO' section displaying the selected reader's details: 'Nombre: Patricia Power', 'Email: patriciapower@ejemplo.com', and 'Teléfono: 123456789'. Below this is the 'LIBRO A PRESTAR (EJEMPLAR)' section with a dropdown menu showing 'Busque o seleccione un ejemplar...'. At the bottom of the form are fields for 'FECHA DE INICIO' (04/05/2026) and 'DEVOLUCIÓN (ESTIMADA)' (dd/mm/aaaa), along with 'Finalizar Registro' and 'Cancelar' buttons. The right panel, titled 'Últimos Préstamos', displays a table of recent loans.

FECHA	LIBRO
04/05/2026	Don Quijote de la Mancha
03/04/2026	Programacion POO de Java 2026
01/04/2026	Libro Genérico Vol 99
23/03/2026	Un Mundo Feliz
20/01/2024	Libro Genérico Vol 59
19/03/2024	Libro Genérico Vol 58
26/03/2024	Libro Genérico Vol 57
18/03/2024	Libro Genérico Vol 56
03/02/2024	Libro Genérico Vol 55

5. Préstamos

Proceso para la salida temporal de material de la biblioteca.

1. Diríjase al módulo de **"Préstamos"**.
2. Haga clic en **"Nuevo Préstamo"**.
3. Busque al **Lector** por su nombre o correo.
4. Seleccione el **Ejemplar** físico del libro que se llevará (debe estar en estado "Disponible").
5. Revise las fechas asignadas automáticamente y haga clic en **"Registrar"**.

Nota: Al registrarse, el estado del ejemplar cambia automáticamente a "Prestado".

1. Se crea el préstamo

localhost:4200 dice
Préstamo registrado exitosamente

Aceptar

Registrar Préstamo

Crear Perfil

BUSCAR LECTOR

Patricia Power

Buscar

LECTOR SELECCIONADO

Nombre: Patricia Power
Email: patriciapower@ejemplo.com
Teléfono: 123456789

LIBRO A PRESTAR (EJEMPLAR)

#105 - Programacion POO de Java 2026

FECHA DE INICIO

05/05/2026

DEVOLUCIÓN (ESTIMADA)

12/05/2026

Finalizar Registro

Cancelar

FECHA	LIBRO	LECTOR
04/05/2026	Don Quijote de la Mancha	KN Karina Nieto
03/04/2026	Programacion POO de Java 2026	JP Juan D. Melénd...
01/04/2026	Libro Genérico Vol 99	JP Juan D. Melénd...
23/03/2026	Un Mundo Feliz	SA Super Admin
20/01/2024	Libro Genérico Vol 59	RR Raúl Romero
19/03/2024	Libro Genérico Vol 58	RF Rubén Fernánd...
26/03/2024	Libro Genérico Vol 57	LL Laura López
18/03/2024	Libro Genérico Vol 56	BO Beatriz Ortiz
03/02/2024	Libro Genérico Vol 55	MV Miguel Vázquez

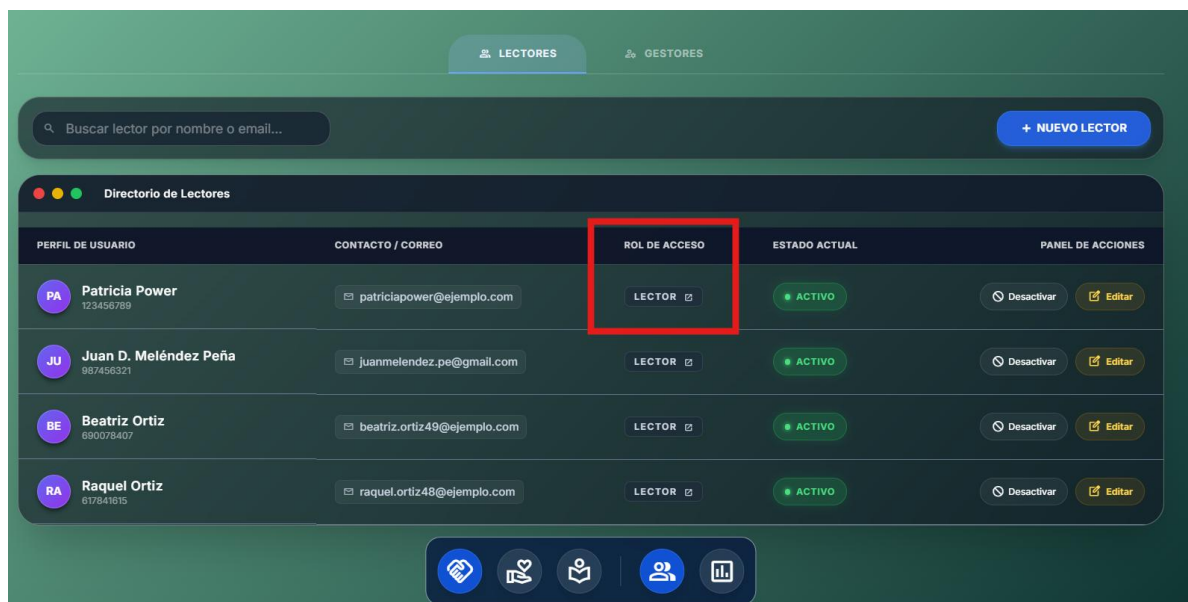
Últimos Préstamos		
FECHA	LIBRO	LECTOR
05/05/2026	Programacion POO de Java 2026	PP Patricia Power
04/05/2026	Don Quijote de la Mancha	KN Karina Nieto
03/04/2026	Programacion POO de Java 2026	JP Juan D. Melénd...
01/04/2026	Libro Genérico Vol 99	JP Juan D. Melénd...
23/03/2026	Un Mundo Feliz	SA Super Admin
20/01/2024	Libro Genérico Vol 59	RR Raúl Romero
19/03/2024	Libro Genérico Vol 58	RF Rubén Fernánd...
26/03/2024	Libro Genérico Vol 57	LL Laura López
18/03/2024	Libro Genérico Vol 56	BO Beatriz Ortiz

6. Devoluciones

Reintegro de un ejemplar al inventario de la biblioteca.

1. Acceda al módulo de **"Devoluciones"**.
2. Utilice el buscador para encontrar el préstamo activo del lector.
3. Haga clic en **"Procesar Devolución"**.
4. Añada observaciones si el libro presenta daños.
5. Haga clic en **"Confirmar"**. El ejemplar pasará nuevamente a estado "Disponible" o "Dañado" según la evaluación.

Se navega en el menu a Gestores -> Lectores -> seleccionar el btn lector de la columna Rol de acceso.



Se abra un panel con las actividades del lector, se verifica que tiene un prestamos. Y si se desea devolver el libro , se realiza click en el btn gestionar devolución, de la columna acción.

PA Patricia Power
patriciapower@ejemplo.com 123456789

← Volver

PRÉSTAMOS (1) DEVOLUCIONES (0) DONACIONES (0)

LIBRO	FECHA INICIO/LÍMITE	ESTADO	ACCIÓN
Programacion POO de Java 2026 ID Ejemplar: 105	5/5/26 5/12/26	ACTIVO	GESTIONAR DEVOLUCIÓN

Iconos de navegación: Home, Buscar, Agregar, Usuario, Analíticas

Se abrirá un formulario con la información del préstamo, solo vamos a agregar una breve descripción de la devolución. Y se confirma la devolución.

FORMULARIO DE DEVOLUCIÓN X CANCELAR

DETALLES DEL PRÉSTAMO ACTIVO

PRESTATARIO (LECTOR)
PA Patricia Power

LIBRO PRESTADO
Programacion POO de Java 2026
Autor: Juan Rulfo
Ejemplar ID: 105 | ISBN: 987-956-321-321

FECHA PRÉSTAMO
May 5, 2026

REGISTRO DE RETORNO

FECHA DE RECEPCIÓN * 04/05/2026

GESTOR RECEPTOR
A Admin Demo (Admin)

NOTAS / ESTADO DEL EJEMPLAR (Opcional)
devolucion sin novedades, todo correcto.

ATRÁS CONFIRMAR DEVOLUCIÓN

Iconos de navegación: Home, Buscar, Agregar, Usuario, Analíticas

Una vez confirmada nos indicará que se ha guardado la devolución. Y volverá a las actividades del usuario y se verifica que el préstamo cambió a devuelto.

PA Patricia Power patriciapower@ejemplo.com 123456789 ← Volver			
PRÉSTAMOS (1) DEVOLUCIONES (1) DONACIONES (0)			
LIBRO	FECHA INICIO/LÍMITE	ESTADO	ACCIÓN
Programacion POO de Java 2026 ID Ejemplar: 105	5/5/26 5/5/26	DEVUELTO	

Podemos navegar a la pestaña devoluciones, y observar los detalles.

PA Patricia Power patriciapower@ejemplo.com 123456789 ← Volver			
PRÉSTAMOS (1) DEVOLUCIONES (1) DONACIONES (0)			
LIBRO DEVUELTO	FECHA RETORNO	GESTOR RECEPTOR	DESCRIPCIÓN
Programacion POO de Java 2026	5/4/26, 12:00 AM	Ana Administradora	devolucion sin novedades, todo correcto.

7. Donaciones

Registro de entrada de nuevos ejemplares aportados por usuarios.

1. Diríjase al módulo de **"Donaciones"**.
2. Seleccione **"Nueva Donación"**.
3. Identifique al usuario donante o seleccione donación anónima.
4. Ingrese los datos correspondientes al libro aportado.
5. Guarde el registro. Esto creará automáticamente el nuevo ejemplar físico en la base de datos.

Registrar Donación

INFORMACIÓN DEL LIBRO

Programación Orientados a Objetos con Java 25 Buscar + Crear Libro

Título: Programación Orientados a Objetos con Java 25 Autor: Juan MP Género: tecnología

DATOS DEL DONANTE

Patricia Power Buscar + Perfil Nuevo

Nombre: Patricia Power Email: patriciapower@ejemplo.com

FECHA CONDICIÓN OBSERVACIONES

04/05/2026 📅 Nuevo 📝 libro nuevo -

Registrar Donación Cancelar

Últimas Donaciones

FECHA	LIBRO / AUTOR	DONANTE
03/04/2026	Programacion de Objetos con Php 8 Autor Desconocido	Juan D. Meléndez...
02/04/2026	Programacion de Objetos con Php 8 Autor Desconocido	Juan D. Meléndez...
01/04/2026	Programacion POO de Java 2026 Juan Ruffo	Juan Morales
21/03/2026	El Principito Antoine de Saint-Exupéry	Super Admin
21/03/2026	El Principito Antoine de Saint-Exupéry	Super Admin
21/03/2026	El Principito Antoine de Saint-Exupéry	Super Admin
21/03/2026	El Principito Antoine de Saint-Exupéry	Super Admin

Alerta de registro

localhost:4200 dice

Donación registrada exitosamente

Aceptar

Registrar Donación

Programación Orientados a Objetos con Java 25 Buscar + Crear Libro

Programación Orientados a Objetos con Java 25 Autor: Juan MP Género: tecnología

Últimas Donaciones

FECHA	LIBRO / AUTOR	DONANTE
03/04/2026	Programacion de Objetos con Php 8 Autor Desconocido	Juan D. Meléndez...
02/04/2026	Programacion de Objetos con Php 8 Autor Desconocido	Juan D. Meléndez...

Se podrá verificar que la donación se guarda automáticamente

Últimas Donaciones		
FECHA	LIBRO / AUTOR	DONANTE
04/05/2026	Programación Orientados a Objetos con Java 25 Juan MP	Patricia Power
03/04/2026	Programacion de Objetos con Php 8 Autor Desconocido	Juan D. Meléndez...

8. Reportes y Métricas Globales

Panel analítico exclusivo para administradores, diseñado para la toma de decisiones.

1. Acceda al módulo **"Reportes"**.
2. Visualizará gráficos y tarjetas de métricas que resumen:
 - Libros más prestados.
 - Porcentaje de ejemplares disponibles vs. prestados.
 - Usuarios con mayores préstamos.
3. Puede aplicar filtros por fechas o tipo de actividad para generar informes precisos de la operación de la biblioteca.



9. Soporte Técnico

Sección de ayuda e información del sistema.

1. Ingrese a la pestaña **"Soporte"** desde el menú de usuario.
2. Encontrará detalles sobre la versión actual del sistema (LiFe System) y las tecnologías utilizadas.
3. Muestra los canales de contacto con el equipo de desarrollo para reporte de bugs o solicitud de mejoras.



6.2 Manual de instalación

Guía de capacitación técnica para la configuración y levantamiento del entorno de desarrollo de LiFe System

1. Requisitos Previos del Sistema

Antes de comenzar con la instalación, asegúrese de que el equipo de desarrollo cuente con las siguientes herramientas instaladas:

- **Java JDK (Versión 17 o 21):** Entorno de ejecución para el servidor Spring Boot.
- **Eclipse IDE:** O cualquier IDE compatible con Java (IntelliJ IDEA, VS Code).
- **Node.js y npm:** Necesarios para compilar y ejecutar el proyecto Angular.
- **PostgreSQL:** Motor de base de datos relacional.
- **pgAdmin 4:** Interfaz gráfica para gestionar la base de datos PostgreSQL.
- **Git:** Para clonar el repositorio del proyecto.

2. Configuración de la Base de Datos

LiFe System requiere una base de datos operativa antes de que el servidor Backend pueda iniciarse.

1. Abra **pgAdmin 4** e inicie sesión en su servidor local de PostgreSQL.
2. Haga clic derecho en **Databases** y seleccione **Create > Database**.
3. Asigne el nombre de la base de datos (por ejemplo, **librarypro_db**) y guárdela.
4. Haga clic derecho sobre la base de datos recién creada y abra la herramienta **Query Tool**.
5. Copie el contenido del archivo **schema.sql** (ubicado en la carpeta **DataBase/** del proyecto) y ejecútelo para crear todas las tablas necesarias.

3. Despliegue del Backend (Java / Spring Boot)

El backend provee la API REST y se conecta directamente a la base de datos PostgreSQL.

Paso 3.1: Importar el proyecto

1. Abra **Eclipse IDE**.
2. Vaya a **File > Import > Maven > Existing Maven Projects**.
3. Seleccione la carpeta `AppBack/libraryPro` de su repositorio y haga clic en *Finish*.
4. Espere a que Maven descargue todas las dependencias listadas en el `pom.xml`.

Paso 3.2: Configurar credenciales

1. En el árbol del proyecto, navegue hasta `src/main/resources/application.properties`.
2. Edite el archivo para que coincida con su configuración local:

```
spring.datasource.url=jdbc:postgresql://localhost:5432/librarypro_db
spring.datasource.username=tu_usuario_postgres spring.datasource.password=tu_contraseña_postgres
```

Paso 3.3: Ejecutar el Servidor

1. Localice la clase principal `AppLibraryPro.java` en `src/main/java/com/ssdk/libraryPro/`.
2. Haga clic derecho sobre el archivo y seleccione **Run As > Java Application** (o *Spring Boot App*).
3. Observe la consola de Eclipse. Debería ver el mensaje de que el Tomcat embebido inició correctamente en el puerto `8080`.

4. Despliegue del Frontend (Angular)

Una vez que el backend y la base de datos están corriendo, podemos levantar la interfaz gráfica de usuario.

1. Abra una terminal de comandos (CMD, PowerShell o Git Bash).
2. Navegue hacia la carpeta del frontend del proyecto:

```
cd ruta/hacia/pr-LibraryPro/AppFron
```

3. Instale las dependencias de Node.js (Esto solo es necesario la primera vez):

```
npm install
```

4. Inicie el servidor de desarrollo de Angular:

```
npm run start
```

Nota: Internamente, esto ejecuta `ng serve --o`.

5. Verificación de Funcionamiento

Para comprobar que toda la arquitectura está comunicándose correctamente:

1. El comando de Angular debió abrir automáticamente su navegador web en `http://localhost:4200`.
2. Si no lo hizo, abra su navegador y visite dicha dirección.
3. Debe visualizar la pantalla de ****Login**** de LiFe System.
4. Intente iniciar sesión con un usuario existente en su base de datos. Si ingresa al sistema sin errores de red, ****la instalación ha sido un éxito.****

En caso de error: Presione `F12` en su navegador, vaya a la pestaña *Consola* o *Red (Network)* y verifique si hay errores CORS o de conexión rechazada. Esto generalmente indica que el Backend de Spring Boot (paso 3) no se inició correctamente.

7 Conclusiones y posibles ampliaciones

Documentación Final: LiFe System

1. Conclusiones

Tras la planificación, diseño y desarrollo del sistema de gestión bibliotecaria **LiFe System**, hemos llegado a las siguientes conclusiones:

- Desarrollar una aplicación completa desde cero es una labor compleja que exige una planificación detallada, un riguroso modelado de datos y una toma de decisiones constante a lo largo del ciclo de vida del software.
- La implementación de tecnologías robustas como Angular y Spring Boot implica una curva de aprendizaje inicial, lo que puede alargar los tiempos de desarrollo y aumentar la probabilidad de errores en las primeras etapas, pero garantiza un resultado final altamente escalable y mantenible.
- La separación de responsabilidades entre el **Backend** (gestión de lógica de negocio y persistencia de datos) y el **Frontend** (interfaz interactiva de usuario) es un estándar arquitectónico fundamental en las aplicaciones modernas, ya que permite que ambos entornos evolucionen de manera independiente.

2. Posibles Ampliaciones (Trabajo Futuro)

El sistema ha sido diseñado con una arquitectura modular que facilita su crecimiento. Algunas mejoras planificadas para versiones posteriores son:

- **Sistema de Notificaciones Automáticas:** Implementar alertas por correo electrónico para notificar a los lectores cuando el plazo de su préstamo esté a punto de expirar o cuando tengan devoluciones atrasadas.
- **Módulo de Reservas y Lista de Espera:** Permitir a los usuarios reservar ejemplares que actualmente se encuentren prestados, generando una cola de prioridad automatizada.
- **Integración con APIs Externas:** Conectar el formulario de registro de "Nuevos Libros" a servicios como *Google Books API* o *OpenLibrary* para autocompletar la metadata (Autor, Género, Portada) ingresando únicamente el código ISBN.
- **Gestión de Sanciones:** Añadir un sistema de control de multas o suspensiones temporales para usuarios que devuelvan los libros en mal estado o fuera del plazo establecido.

8 Bibliografía

Bibliografía y Herramientas

A continuación se detallan las principales tecnologías, frameworks y recursos documentales utilizados durante la construcción de LiFe System:

- **Angular.** (s. f.). The modern web developer's platform. <https://angular.io/>
- **Spring Boot.** (s. f.). Spring Boot Framework. <https://spring.io/projects/spring-boot>
- **Java.** (s. f.). Java Programming Language. <https://www.java.com/>
- **PostgreSQL.** (s. f.). The World's Most Advanced Open-Source Relational Database. <https://www.postgresql.org/>
- **pgAdmin.** (s. f.). PostgreSQL Tools. <https://www.pgadmin.org/>
- **TypeScript.** (s. f.). JavaScript With Syntax For Types. <https://www.typescriptlang.org/>
- **Tailwind CSS.** (s. f.). Rapidly build modern websites without ever leaving your HTML. <https://tailwindcss.com/>
- **Eclipse IDE.** (s. f.). The Eclipse Foundation. <https://www.eclipse.org/ide/>
- **Figma.** (s. f.). The collaborative interface design tool. <https://www.figma.com/>

9 Anexo